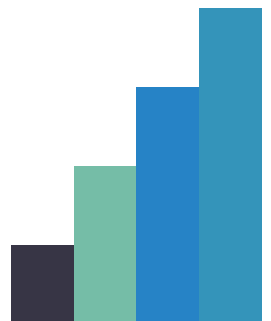




Prácticas de Álgebra Lineal

<https://es.mathworks.com/>

**GRADO EN INGENIERÍA INFORMÁTICA
EN TECNOLOGÍAS DE LA INFORMACIÓN**



Índice general

I	Introducción a MatLab	5
1.	Introducción a MATLAB	7
1.1.	Interfaz de MATLAB	7
1.2.	Trabajo en la ventana de comandos	8
1.3.	Variables	9
1.3.1.	Reglas para nombrar variables	10
1.3.2.	Cómo borrar variables	10
1.3.3.	Algunas variables predefinidas en MATLAB	11
1.3.4.	Constantes y variables simbólicas	11
1.3.5.	Información sobre las variables	13
1.4.	Archivos de órdenes	14
1.5.	Vectores y matrices	15
1.5.1.	Construcción de vectores y matrices	15
1.5.2.	Acceso y modificación en matrices	16
1.5.3.	Definición de matrices por bloques	17
1.5.4.	Operaciones con vectores y matrices	18
1.5.5.	Funciones básicas que actúan sobre vectores y matrices	19
II	Conjuntos y Número complejos	21
2.	Conjuntos	23
3.	Números complejos	27
3.1.	La unidad imaginaria y los números complejos en MATLAB	27
3.2.	Las operaciones	28
3.3.	Funciones básicas	29
3.4.	Representación gráfica en el plano	30

III	Sistemas de Ecuaciones Lineales	33
4.	Sistemas de ecuaciones lineales	35
4.0.1.	Clasificación de sistemas	35
4.0.2.	Resolución de sistemas	40
IV	Espacios Vectoriales	47
5.	Espacios vectoriales	49
5.1.	Dependencia e independencia lineal	49
5.2.	Base y dimensión	50
5.3.	Subespacios vectoriales. Suma e intersección de subespacios	51
5.4.	Cambio de base	55
V	Aplicaciones Lineales y Diagonalización	59
6.	Aplicaciones lineales	61
6.1.	Representación de una aplicación lineal	61
6.2.	Núcleo e imagen de una aplicación lineal	63
6.3.	Cambio de bases en una aplicación lineal	65
7.	Diagonalización	71
7.1.	Valores y vectores propios	71
7.1.1.	Propiedades básicas	73
7.2.	Diagonalización de endomorfismos y matrices	76
7.2.1.	Aplicaciones de la diagonalización	82
VI	PRUEBA	89

Parte I

Introducción a MatLab

Capítulo 1

Introducción a MATLAB

MATLAB es un sistema de cálculo científico que ofrece un entorno de desarrollo integrado, IDE, y posee un lenguaje de programación propio (lenguaje M). La primera versión de MATLAB data de los años 70, y fue diseñado como herramienta de apoyo para cursos de *Teoría de matrices*, *Álgebra Lineal* y *Análisis numérico*. El nombre MATLAB es un acrónimo de ‘MATrix LABoratory’. Hoy en día es un potente programa con un entorno agradable que incluye herramientas de cálculo científico y técnico y de visualización gráfica, así como un lenguaje de programación de alto nivel.

1.1. Interfaz de MATLAB

Cuando abrimos MATLAB, como en todos los programas que trabajan bajo Windows, nos aparecen en pantalla varios iconos de acceso rápido, tres pestañas (HOME, PLOTS y APPS) y varias ventanas. La distribución de ventanas en la pantalla depende de la configuración elegida por el usuario. Si tenemos activa la pestaña HOME, pinchamos en el acceso directo *Layout, Default* aparecen en pantalla las subventanas que describiremos a continuación.

La ventana *Current Folder* nos indica el directorio en el que estamos trabajando y nos muestra los ficheros que hay en dicha carpeta.

La ventana *Command Window* es la que utilizamos para escribir las instrucciones a continuación del siguiente *prompt*:

```
>>
```

Por ejemplo, podemos teclear

```
>> 3+5
```

y, luego, para obtener los resultados es necesario teclear la tecla de *cambio de línea* $\leftrightarrow$, que nos devuelve

```
ans =  
      8
```

En la ventana *Workspace* nos aparecen las variables que vamos declarando y en la ventana *Command History* un historial con los comandos que hemos escrito.

Pinchando en la barra de herramientas sobre la palabra **help** podemos acceder a diversos menús de ayuda. Si queremos obtener información sobre un determinado comando basta escribirlo en la ventana de comandos precedido de la orden **help**. Por ejemplo, escribíbase

```
>> help limit
```

y se obtendrá toda la información necesaria para calcular límites.

1.2. Trabajo en la ventana de comandos

Para realizar los cálculos numéricos elementales con MATLAB es suficiente conocer la sintaxis de las distintas operaciones:

Suma	Resta	Multiplicación	División	Potenciación
+	-	*	/	^

Las operaciones se evalúan de izquierda a derecha. La operación potencia tiene el orden de prioridad más alto, seguida de la multiplicación y división, que se encuentran en el mismo orden de prioridad y seguidas finalmente por la suma y la resta, ambas con la misma prioridad. Para alterar la ordenación se pueden utilizar paréntesis en la forma habitual.

Ejemplo 1.1 Halle el valor de $5^2 - 17 \cdot 2 + 7/5 - \sqrt{12}$ □

```
>> 5^2-17*2+7/5-(12)^(1/2)
ans =
-11.0641
```

Ejemplo 1.2 Obsérvese la diferencia entre las siguientes operaciones:

$$3/3+4; \quad 3/(3+4); \quad 3^2 \cdot 3; \quad 3^{2 \cdot 3}$$

□

```
>> 3/3+4
ans =
5
>> 3/(3+4)
ans =
0.4286
>> 3^2*3
ans =
27
>> 3^(2*3)
ans =
729
```

Al trabajar en la ventana de comandos conviene tener en cuenta:

- la utilidad de las flechas situadas en el teclado entre el teclado numérico y alfanumérico. Las ordenes que vamos introduciendo en la ventana de comandos se van almacenando y pueden recuperarse con la tecla $\uparrow$. Una vez que tenemos la línea deseada, podemos modificarla, desplazando el cursor sobre ella con las teclas $\leftarrow$ y $\rightarrow$ y ejecutarla de nuevo. También podríamos recuperar una orden ya introducida seleccionándola con el ratón en la subventana correspondiente a *Command History* pinchando dos veces sobre ella, con lo que se ejecuta la orden o bien arrastrándola con el ratón hasta la ventana de comandos. Por contra, no es posible situarse en una línea anterior, hacer una corrección y volver a ejecutarla.

- El **%** sirve para insertar comentarios. Todo lo que escribamos detrás no será tenido en cuenta por MATLAB.

```
>> 2+3 %hemos hecho una suma
ans =
    5
```

- Para finalizar o separar órdenes se utiliza *el salto de línea*, *el símbolo “,”* y *el símbolo “;”*.

```
>> 5+4; 2+9; 9^2,3+2
ans =
    81
ans =
    5
```

- Observa que, si inmediatamente detrás de una orden se pone un punto y coma, la orden se ejecuta, pero el resultado no aparece en pantalla.
- La orden **clc** sirve para situar el prompt >> en la primera línea. Limpia la pantalla pero esta acción no borra de la memoria nada que haya sido creado anteriormente.

Ejercicio 1.1

1. Obsérvese la diferencia entre las siguientes operaciones:

$$3^2 - 5 \left(2 - \frac{3}{4} \cdot 7 \right); \quad 3^2 - 5 \cdot 2 - \frac{3}{4 \cdot 7}$$

2. Hállese el valor de $7^2 - 17 \cdot 2 + 7/5 - \sqrt{15}$, recuperando la orden del apartado 2 con la tecla $\uparrow$ y modificando los números correspondientes.
3. Bórrese la pantalla con la orden **clc**.

1.3. Variables

Una variable es un nombre al cual se le asigna un valor. El símbolo **=** es el utilizado para la asignación de valores a las variables. Por ejemplo:

```
>> x=3
x =
    3
```

El nombre de la variable siempre debe ir a la izquierda del símbolo **=** y el valor a la derecha.

Es muy importante tener en cuenta que el valor que se asigna a las variables es permanente. Se guardará hasta que la variable sea borrada o se le asigne un nuevo valor.

```
>> x^2
ans =
    9
```

Asignamos un nuevo valor a **x** :

```
>> x=7
x =
    7
>> x^2
ans =
   49
```

1.3.1. Reglas para nombrar variables

Las reglas que se utilizan para nombrar las variables son las siguientes:

- MATLAB distingue entre letras mayúsculas y minúsculas. Las variables **area**, **Area**, **AREA**, **arEa** serían variables distintas.
- El nombre de una variable puede contener un máximo de 31 caracteres ignorándose los posteriores.
- El nombre de una variable debe empezar necesariamente por una letra, aunque puede contener letras, números y el guión de subrayado. Nunca puede contener operadores (+,*,...), espacios en blanco ni signos de puntuación.
- No deben nombrarse variables con nombres con significado específico de funciones o variables predefinidas en MATLAB, por ejemplo **cos**=3 construiría una variable **cos**=3 cuyo valor es **3**, y a partir de este momento no podríamos calcular el coseno de un ángulo hasta que no borrásemos la variable **cos**=3.

Ejercicio 1.2

1. Ejecútese la orden `>> 34=b` y obsérvese lo que ocurre.
2. Asígnese a los nombres **x**, **z** y **area** los valores 3, 5 y 27 respectivamente.
3. Calcúlese el valor **x+area**.
4. Asígnese los nombres **a1** y **1a** los valores 8 y 88 y obsérvese lo que ocurre.
5. Calcúlese $5 + \sqrt{2}$ y ejecútese a continuación la orden **ans**. Obsérvese que en **ans** se almacena el resultado de la última orden ejecutada.

1.3.2. Cómo borrar variables

La orden **clear** se utiliza para borrar todas las variables definidas hasta el momento; si a la orden se le añade una lista de variables (separadas por espacios en blanco) sólo se borrarán las variables de la lista.

Ejercicio 1.3

1. Bórrese la pantalla con la orden **clc**. Obsérvese que siguen en memoria todas las variables.
2. Bórrese las variables **x** y **area**.
3. Bórrese el resto de las variables.

También se pueden borrar variables pinchando con el botón derecho del ratón sobre la variable en la ventana *Workspace* y seleccionando *Delete*.

1.3.3. Algunas variables predefinidas en MATLAB

Algunas variables ya están definidas en MATLAB:

Nombre	Significado
ans	Almacena el último resultado no asignado a una variable
eps	Epsilon de la máquina
pi	π
i y j	Unidad imaginaria
inf	∞
NaN	No es un número
date	Fecha

El épsilon de la máquina es el número positivo más pequeño que sumado a 1 genera un número mayor que 1 en el ordenador, (en un PC **eps**=2.220446049250313e-016) y **NaN** (*Not a Number*) representa una expresión indeterminada, como puede verse en el siguiente ejemplo:

```
>> (2-2)/(3-3)
```

Ejercicio 1.4

1. Calcúlese $5 + \pi$.
2. Defínase una variable con el nombre **pi** y almacénese en ella el valor 6.
3. Calcúlese $5 + \pi$. Nótese que no deben nombrarse variables con nombres con significado específico de funciones o variables predefinidas en MATLAB. (Tras el apartado 2 no podríamos trabajar con el número π)
4. Bórrese la variable **pi** y obsérvese que hemos recuperado su valor original.

1.3.4. Constantes y variables simbólicas

Los cálculos en MATLAB, por defecto, se realizan en formato numérico. Si efectuamos, por ejemplo, operaciones como $1/2 + 1/5$, o $(\pi^2 - 1)/(\pi - 1)$, nos devuelve los valores aproximados, 0.7000 y 4.1416, en vez de los resultados exactos $7/10$ y $\pi + 1$, respectivamente.

Por otra parte, hay cálculos que se realizan habitualmente en Matemáticas y que no son posibles con las órdenes de MATLAB estudiadas hasta el momento. Por ejemplo:

- $(a+b)(a-b) = a^2 - b^2$
- $\int (2x) dx = x^2$

no se pueden realizar porque MATLAB no trabaja con variables que no tengan valores numéricos asignados.

Para solventar este tipo de problemas, podemos utilizar la herramienta *cálculo simbólico* de MATLAB. A las expresiones sobre las que se realiza el cálculo simbólico se las denomina *expresiones simbólicas* y a las constantes y variables que aparecen en ellas *constantes y variables simbólicas*.

Para declarar variables simbólicas utilizaremos la orden **syms**, escribiendo a continuación, y separadas por espacios en blanco, las variables que se deseen.

```
>> syms y z % Crea las variables simbólicas y, z
```

Cuando una expresión contiene variables simbólicas, dicha expresión también lo es.

```
>> syms x y
>> f(x)= x^2+2*x+1 % Crea la función simbólica f
f(x) =
  x^2+2*x+1
>> g=z^2+y % Crea la expresión simbólica g
g = z^2 + y
```

y ahora podemos hacer operaciones simbólicas con f y g , como derivadas o integrales, por ejemplo.

Para derivar, se utiliza el comando **diff**. Cuando se trabaja con funciones de una variable, basta utilizar la orden **diff(función)** para hallar la primera derivada y **diff(función,n)** para calcular la derivada n -ésima:

```
>> f1(x)=diff(f(x))
f1(x) =
  5*x^4
>> f3(x)=diff(f(x),3)
f3(x) =
  60*x^2
```

Si la función es de varias variables, lo más cómodo es especificar la variable de derivación, utilizando **diff(función,variable)** y **diff(función,variable,n)**, para calcular la parcial primera de una función y la parcial n -ésima, respectivamente:

```
>> g_x(x,y)=diff(g(x,y),x)
g_x(x, y) =
  2*x
>> g_y2(x,y)=diff(g(x,y),y,2)
g_y2(x, y) =
  -6*y
```

Para integrar, se usa la orden **int**. Centrándonos en el caso de funciones de una variable con **int(función)**, hallamos una primitiva de una función y con **int(función,límite_inferior,límite_superior)**, calculamos la integral definida de una función:

```
>> int(f(x))
ans =
  x^6/6
>> s=int(f(x),0,1)
s =
  1/6
```

También podemos hallar el valor de una función simbólica en uno o más puntos, sin más que ejecutar, por ejemplo:

```
>> f(2)
ans =
  32
>> f(-2:2)
ans =
  [-32, -1, 0, 1, 32]
```

```
>> g(2,1)
ans =
3
```

Como se puede ver, todas las operaciones se hacen de forma simbólica y, en caso de que deseemos obtener un **cálculo en forma decimal**, debemos utilizar el comando **double** y para convertir una **constante numérica en simbólica se utiliza el comando sym**:

```
>> a=sym(pi) % Almacena en a la constante simbólica pi
a = pi

>> n=double(a) % Aproxima pi con 16 dígitos y lo almacena en n
n =
    3.1416
```

Por último, la orden **vpa(s,d)**, evalúa s utilizando d dígitos

```
>> s=sym(sqrt(2))
s = 2^(1/2)
>> s=vpa(s,6)
s = 1.41421
```

Ejercicio 1.5

1. Defínase la expresión simbólica $f = x^2 + y^2$.
2. Constrúyanse las constantes simbólicas $a = 1/4$, y $b = \sqrt{2}$.
3. Sustitúyanse en f , x e y por a y b .

1.3.5. Información sobre las variables

Para obtener información sobre las variables definidas en una sesión de trabajo se utilizan las órdenes **who** y **whos**. La primera muestra las variables que tienen valores asignados, la segunda nos da además información sobre el tamaño y el tipo de dato.

```
>> who
>> whos
```

Puede observarse que MATLAB utiliza los escalares como matrices 1×1 .

Ejercicio 1.6

1. Ejecútense las órdenes **who** y **whos** y obsérvese la información que proporcionan.
2. Nótese que en la ventana Workspace se almacenan las variables que vamos creando.

1.4. Archivos de órdenes

Escribir los comandos directamente en la ventana de trabajo de MATLAB resulta cómodo para ejecutar unas pocas órdenes. Sin embargo, cuando queremos guardar y ejecutar un gran número de instrucciones de forma consecutiva, necesitamos crear un *programa*.

Los programas en MATLAB se crean con ficheros de extensión **.m**.

Seguiremos los pasos descritos en el ejercicio siguiente:

Ejercicio 1.7

1. Píñchese con el ratón el primer icono de la barra de herramientas (“New Script”) o bien el segundo (“New Live Script”).

Cabe reseñar que, salvo que la última configuración utilizada sea diferente, el editor aparecerá como una ventana independiente. A veces es cómodo trabajar con el editor integrado dentro del programa principal, para ello deberemos pinchar en el penúltimo icono del grupo de iconos de la esquina superior derecha (▼), se abre un menú y se pincha Dock Editor (para deshacerlo se pincharía en Undock).

2. En el fichero que se ha abierto escríbanse las siguientes órdenes:

```
x=5
y=7
S=x+y
```

3. Para ejecutar las órdenes escritas antes de salvar el fichero, selecciónese con el ratón y púlsese a continuación **F9**. Obsérvese que las órdenes se ejecutan en la ventana de comandos.
4. Al fichero aun abierto añádase la orden

```
P=x*y
```

5. Sálvese el fichero, en el directorio elegido para trabajar, pinchando: Save --> Save As y dándole el nombre **Ejercicio0_7**.
6. Compruébese que el nombre del fichero **Ejercicio0_7.m** aparece en la ventana de la izquierda (Current Folder).
7. Una vez salvado el fichero ejecútese secuencialmente todas sus órdenes escribiendo en la ventana de comandos su nombre (sin la extensión **.m**) y pulsando **enter** a continuación.

Ejercicio 1.8

1. Constrúyase un fichero de nombre **Ejercicio0_8** con la siguiente secuencia de órdenes:

```
% Vamos a obtener el área de un círculo
r=input('Introduce el radio de la circunferencia ');
a=r^2*pi;
disp('El área es:')
disp(a)
```

2. Una vez salvado el fichero, ejecútese.
3. Obsérvese que la orden $a=r^2\pi$, se ejecuta pero no sale en pantalla por ir seguida de ;.
4. Obsérvese que los comentarios precedidos de % no salen en pantalla.
5. Escríbase en la ventana de comandos **help Ejercicio0_8** y aparecerá la descripción que hemos puesto en la primera línea.

Nota 1.1 Para nombrar ficheros se siguen las mismas reglas que para nombrar variables.

1.5. Vectores y matrices

El tipo básico de dato con el que MATLAB trabaja es la matriz, incluso los escalares son considerados como matrices 1×1 , por lo que es esencial familiarizarse con esta sección.

1.5.1. Construcción de vectores y matrices

Los **vectores** se introducen escribiendo cada una de sus coordenadas entre corchetes, separadas por un espacio en blanco o bien separadas por comas.

Si separamos las componentes mediante punto y coma, obtendremos un vector columna.

Ejemplo 1.3

```
>> v=[1 3 pi 1/3]
```

```
>> v=[1,3,pi,1/3]
```

```
>> w=[1;3;pi;1/3]
```

□

No obstante, existen formas de introducir vectores, cuando sus coordenadas guardan alguna relación entre sí:

Ejemplo 1.4

```
>> v=[1:0.3:2]
```

```
>> v=(1:-0.4:-0.8)
```

```
>> v=1:4
```

```
>> v=linspace(1,2,5)
```

□

Para construir una **matriz**, se deben tener en cuenta las siguientes reglas:

- Los **elementos se introducen entre corchetes**.
- Los **elementos de una misma fila se introducen como un vector fila**, separados por comas o por espacios en blanco.
- **Para separar filas se utiliza el punto y coma o el cambio de línea**:

Ejemplo 1.5 ¹

```
>> A=[1 2 3; 3,1,2;1 1 0]
```

```
>> BX=[1 2 3; 2, 4,5  
3 5 6]
```

¹El cambio de línea se consigue con las teclas May+Enter



Pueden definirse ciertas matrices con las siguientes órdenes:

Función	Salida
ones(n)	Matriz cuadrada $n \times n$ de unos.
ones(m,n)	Matriz $m \times n$ de unos.
zeros(n)	Matriz cuadrada $n \times n$ de ceros.
zeros(m,n)	Matriz $m \times n$ de ceros.
eye(n)	Matriz identidad $n \times n$.
eye(m,n)	Matriz $m \times n$ con unos en la diagonal principal y el resto ceros.
toeplitz(v)	Matriz simétrica de diagonales constantes y el vector v como primera fila y primera columna.
toeplitz(v,w)	Matriz de diagonales constantes, el vector v como primera fila y el vector w como primera columna.
diag(v)	Matriz con el vector v en la diagonal principal y el resto ceros.
rand(n)	Matriz $n \times n$ con elementos de valor aleatorio entre 0 y 1 (distribución uniforme).
randn(n)	Matriz $n \times n$ cuyos elementos siguen una distribución normal (media 0 y varianza 1).

1.5.2. Acceso y modificación en matrices

Una vez definida una matriz o un vector, se puede acceder a sus elementos o submatrices con las órdenes:

Función	Salida
v(i)	Coordenada i del vector v
A(i,j)	Elemento de la matriz A que ocupa la posición i,j
A(:,j)	Columna j de la matriz A
A(:,end)	Columna última de la matriz A
A(i,:)	Fila i de la matriz A
A(end,:)	Fila última de la matriz A
A(v,w)	Submatriz de A que contiene las coordenadas indicadas por las filas indicadas en v y las columnas indicadas en w
A(i,:)=[]	Elimina la fila i de la matriz A
A(:,j)=[]	Elimina la columna j de la matriz A
	Nota: [] representa la matriz vacía (<i>empty matrix</i>)

Ejemplo 1.6 Recuperar con el ratón en la subventana correspondiente a Command History el vector v y la matriz A (pinchar dos veces sobre v y A) y teclear:

```
>> v(2)
>> A(1,2)
```

devuelve respectivamente la segunda componente de v y el elemento de la primera fila, segunda columna de A .

```
>> v([1,3])
>> A([2,1],1)
>> A(:,2)
```

nos devuelven respectivamente las componentes primera y tercera de v , los elementos de las filas segunda y primera, de la primera columna de A y los elementos de la segunda columna de A . □

Haciendo uso de estas órdenes pueden introducirse matrices y vectores elemento a elemento. Al asignarle un valor a una posición, se construye la matriz o vector de menor tamaño que contiene los elementos introducidos y hace ceros los no asignados.

Ejemplo 1.7

```
>> B(5)=3  
>> B(2,4)=5
```

□

O bien se puede utilizar para modificar una posición de una matriz predefinida o de varias posiciones simultáneamente.

Ejemplo 1.8

```
>> A=[2 7;3 9]  
>> A(1,2)=5  
>> A=ones(5)  
>> A(3,:)=20
```

□

También pueden eliminarse filas y columnas de matrices dadas.

Ejemplo 1.9

```
>> A=[ 2 4 6; 3 5 8]  
>> B=A  
>> B(:,2)=[]
```

□

1.5.3. Definición de matrices por bloques

Dadas dos matrices A y B con el mismo número de filas, se puede definir una matriz C formada por todas las columnas de A y de B y, análogamente, se puede definir una matriz a partir de otras dos con el mismo número de columnas.

Ejemplo 1.10

```
>> A=ones(3), B=eye(3,2), C=[A B]  
  
>> A=zeros(2,3), B=ones(3), C=[A;B]
```

□

1.5.4. Operaciones con vectores y matrices

Si A y B son matrices con las dimensiones adecuadas y λ es un escalar, las operaciones habituales se efectúan con las siguientes órdenes:

Operación	Resultado
$A+B$	Suma A y B
$A-B$	Resta B de A
$A*B$	Multiplica A por B
A/B	Calcula AB^{-1}
$A \setminus B$	Calcula $A^{-1}B$
$\lambda * A$	Multiplica todos los elementos de A por λ
A^n	Eleva la matriz A al entero n
$A.'$	Calcula la traspuesta de A
A'	Calcula la traspuesta de la conjugada de A

Ejemplo 1.11

```
>> A=[1 5 7;2 6 9], B=ones(3), C=A*B, C=A+C, C=B^2, C=A'
```

□

Además de las operaciones mencionadas, en MATLAB se definen otras operaciones a las que llamaremos **operaciones elemento a elemento**:

Operación	Resultado
$\lambda + A$	Suma a cada elemento de A el escalar λ
$A.*B$	Calcula una matriz que en la posición (i, j) contiene el producto $a_{ij}b_{ij}$ de los elementos que en A y B ocupan dicha posición
$A./B$	Calcula una matriz que en la posición (i, j) contiene el cociente a_{ij}/b_{ij} de los elementos que en A y B ocupan dicha posición
$a./B$	Calcula una matriz que en la posición (i, j) contiene el cociente a/b_{ij} , siendo a un número real fijo.
$A.^n$	Eleva cada elemento de la matriz A al entero n
$A.^B$	Calcula una matriz que en la posición (i, j) contiene $a_{ij}^{b_{ij}}$
$a.^B$	Calcula una matriz que en la posición (i, j) contiene $a^{b_{ij}}$, siendo a un número real fijo.

Ejemplo 1.12

```
>> A=[1 5 7;2 6 9], B=2+ones(3)
>> C=A*B
>> C=A.*B % Da error
>> C=A.*A, C=B.^2, C=B^2, C=A.^2, C=2.^A, C=1./A
```

□

1.5.5. Funciones básicas que actúan sobre vectores y matrices !!!!!!!

Las siguientes funciones permiten obtener información sobre las matrices o vectores que tienen como argumentos de entrada:

Función	Salida
size(A)	Vector con las dimensiones de la matriz A
size(A,1)	Número de filas de la matriz A
size(A,2)	Número de columnas de la matriz A
diag(A)	Vector cuyos elementos son los de la diagonal principal de A
rank(A)	Rango de la matriz A
det(A)	Determinante de la matriz A
inv(A)	Inversa de la matriz A cuando A es inversible
rref(A)	Matriz escalonada reducida de A
trace(A)	Traza (suma de los elementos de la diagonal principal o de los autovalores) de la matriz A
null(A)	Matriz cuyas $n - r$ primeras columnas forman una base ortogonal del espacio nulo de A , es decir, del subespacio de soluciones de un sistema homogéneo de matriz A
orth(A)	Matriz cuyas r primeras columnas forman una base ortogonal del espacio de columnas de A
colspace(A)	Función simbólica cuyas columnas forman una base del espacio de columnas de A
triu(A)	Matriz triangular superior cuyos elementos sobre la diagonal principal son los de la matriz A
tril(A)	Matriz triangular inferior cuyos elementos bajo la diagonal principal son los de la matriz A

Finalmente, conviene resaltar que las funciones matemáticas habituales que el programa tiene predefinidas (trigonométricas, exponenciales, etc.) también pueden utilizarse sobre matrices o vectores, con la particularidad de que en tal caso actúan elemento a elemento.

Ejemplo 1.13

```
>> A=[0 pi;pi/2 3*pi/2];
>> cos(A) % matriz cuyos elementos son los cosenos de los elementos de A
ans =
    0    0.0000
   1.0000   -1.0000

>> B=[1+i,-1,1;2+2i,0,-5i];
>> C=abs(B) % matriz cuyos elementos son los módulos de los elementos de B
C =
   1.4142    1.0000    1.0000
   2.8284     0    5.0000

>> D=imag(B) % matriz cuyos elementos son las partes imaginarias de los B
D =
    1     0     0
    2     0    -5
```



Parte II

Conjuntos y Número complejos

Capítulo 2

Conjuntos

En MATLAB se pueden realizar algunas operaciones con conjuntos. Los elementos de los conjuntos se almacenarán en vectores. Las principales funciones que realizan operaciones sobre conjuntos son las siguientes:

Función	Salida
c=unique(u)	devuelve un vector c con las mismas coordenadas que u , pero sin repeticiones, por lo tanto, devuelve el conjunto de los elementos de u
c=intersect(u,v)	Devuelve la intersección de u y v conjuntos
c=ismember(u,v)	Devuelve un vector c del mismo tamaño que u , que contiene un 1 en la posición en las que el elemento está en v y un 0 en el resto
c=setdiff(u,v)	Devuelve un vector u con los elementos de u que no están en v , sin repeticiones y ordenados.
c=setxor(u,v)	Devuelve el vector c que contiene los elementos de la unión de u y v que no están en su intersección, sin repetición y ordenados.
c=union(u,v)	Devuelve el vector c que contiene los elementos de la unión de u y v , sin repetición y ordenados.
c=sort(u,'direccion')	devuelve el vector u ordenado. Si direccion=ascend en orden creciente, si direccion=descend en orden decreciente. Si el vector u contiene letras y números, se consideran menores los números que las letras. Si no se pone la segunda entrada, toma direccion=ascend .

Ejemplo 2.1 Sea *A* el conjunto de números múltiplos de 3 menores de 50 y *B* el de los números de la forma $2 + 8n$ menores de 50. □

```
A = 3:3:50;
B = 2:8:50;
c = intersect(A,B)
```

```
c = 1x2
    18    42
```

```
d = setdiff(A,B)
```

```
d = 1x14
     3     6     9    12    15    21    24    27    30    33    36    39    45    48
```

```
e = setxor(A,B)
```

```
e = 1x19
      2   3   6   9  10  12  15  21  24  26  27  30  33  34  36  39
45  48  50
```

Ejemplo 2.2 *Dados los conjuntos*

$$U = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$$

$$A = \{1, 4, 7, 10\}$$

$$B = \{1, 2, 3, 4, 5\}$$

$$C = \{2, 4, 6, 8\}$$

Calcula los conjuntos siguientes:

I. $A \cap B$

II. $A \cup B$

III. $A \setminus B$

IV. $\mathcal{C}_U B \cap (C \setminus A)$

V. $\mathcal{C}_U (A \cap B) \cup C$

VI. $(A \cap B) \cup C$

VII. $(A \cup B) \setminus (C \setminus B)$

y comprueba las igualdades:

$$(A \cup B) \cap C = (A \cap C) \cup (B \cap C) \quad (2.0.1)$$

$$(A \cap B) \cup C = (A \cup C) \cap (B \cup C) \quad (2.0.2)$$

□

Introducimos los datos:

```
U = 1:10
```

```
U = 1x10
      1   2   3   4   5   6   7   8   9  10
```

```
A = 1:3:10
```

```
A = 1x4
      1   4   7  10
```

```
B = 1:5
```

```
B = 1x5
      1   2   3   4   5
```

```
C = 2:2:8
```

```
C = 1x4
      2   4   6   8
```

I.

```
intersect(A,B)
```

```
ans = 1x2
      1      4
```

II.

```
union(A,B)
```

```
ans = 1x7
      1      2      3      4      5      7      10
```

III.

```
setdiff(A,B)
```

```
ans = 1x2
      7      10
```

IV.

```
intersect(setdiff(U,B),setdiff(C,A))
```

```
ans = 1x2
      6      8
```

V.

```
union(setdiff(U,intersect(A,B)),C)
```

```
ans = 1x9
      2      3      4      5      6      7      8      9      10
```

VI.

```
union(intersect(A,B),C)
```

```
ans = 1x5
      1      2      4      6      8
```

VII.

```
setdiff(union(A,B),setdiff(C,B))
```

```
ans = 1x7
      1      2      3      4      5      7      10
```

Igualdad (2.0.1)

```
ismember(intersect(union(A,B),C),union(intersect(A,C),intersect(B,C)))
```

```
ans = 1x2 logical array
      1      1
```

Igualdad (ref2)

```
ismember(union(intersect(A,B),C),intersect(union(A,C),union(B,C)))
```

```
ans = 1x5 logical array
      1      1      1      1      1
```


Capítulo 3

Números complejos

3.1. La unidad imaginaria y los números complejos en MATLAB

En MATLAB, la unidad imaginaria $\sqrt{-1}$ se representa con las letras **i** o **j**

```
>> i^2
ans =
    -1
>> j^2
ans =
    -1
>> sqrt(-1)
ans =
    0 + 1.0000i
```

Los números complejos se escriben de la siguiente manera:

$$\begin{array}{ll} \text{complejo} = a + bi & \text{complejo} = a + bj \\ \text{complejo} = a + b*i & \text{complejo} = a + b*j \end{array}$$

La utilización del operador de multiplicación `''*` es necesaria en caso que la parte imaginaria, **b**, se obtenga como resultado de la aplicación de una función o expresión más compleja que un número.

Ejemplo 3.1

```
>> c=3+4i
c =
    3.0000 + 4.0000i
>> 1+sqrt(9)i
1+sqrt(9)i
    ↑
```

Error: Invalid expression. Check for missing multiplication operator, missing or unbalanced delimiters, or other syntax error. To construct

matrices, use brackets instead of parentheses.

Did you mean:

```
>> 1+sqrt(9)*i
```

y al darle al enter obtenemos

```
ans =  
    1.0000 + 3.0000i
```

De la misma forma obtenemos un error si escribimos

```
>> (1-3/4)i  
(1-3/4)i  
      ↑
```

Error: Invalid expression. Check for missing multiplication operator, missing or unbalanced delimiters, or other syntax error. To construct matrices, use brackets instead of parentheses.

En este caso no nos hace la sugerencia ya que hay varias opciones correctas e igualmente probables. Deberíamos escribir:

```
>> (1-3/4)*i  
ans =  
    0 + 0.2500i
```

□

3.2. Las operaciones

Las operaciones con complejos, y sus inversas, se realizan igual que con números reales:

```
(1 - i) + (3 + 2i) % suma de números complejos
```

```
ans = 4.0000 + 1.0000i
```

```
(1 - i) - (3 + 2i) % resta de números complejos
```

```
ans = -2.0000 - 3.0000i
```

```
(1 - i) * (3 + 2i) % producto de números complejos
```

```
ans = 5.0000 - 1.0000i
```

```
(1 - i) / (3 + 2i) % cociente de números complejos
```

```
ans = 0.0769 - 0.3846i
```

```
(1 + i) ^ 2 % potencia
```

```
ans = 0.0000 + 2.0000i
```

3.3. Funciones básicas

Para operar con complejos son útiles las siguientes funciones:

Orden	Salida
<code>isreal(z)</code>	0 si z tiene parte imaginaria 1 en caso contrario
<code>real(z)</code>	Parte real de z
<code>imag(z)</code>	Parte imaginaria de z
<code>abs(z)</code>	Módulo de z
<code>conj(z)</code>	Conjugado de z
<code>angle(z)</code>	Argumento, en radianes, que se encuentra en el intervalo $[-\pi, \pi]$

Ejemplo 3.2 Dado el número complejo $z = 1 + i$ obtener su parte real, su parte imaginaria, su módulo, su forma polar con el ángulo medido en grados (recordemos que la orden devuelve radianes) y su conjugado. Comprobar que la orden `angle(z)` equivale a la forma de obtenerla "en papel". $\square$

$$z = 1 + i$$

$$z = 1.0000 + 1.0000i$$

$$a = \text{real}(z)$$

$$a = 1$$

$$b = \text{imag}(z)$$

$$b = 1$$

$$\text{isreal}(z)$$

$$\text{ans} = 0$$

$$\text{abs}(z)$$

$$\text{ans} = 1.4142$$

$$\text{angle}(z)$$

$$\text{ans} = 0.7854$$

$$\text{angle}(z) * 180 / \pi$$

$$\text{ans} = 45$$

$$z'$$

$$\text{ans} = 1.0000 - 1.0000i$$

3.4. Representación gráfica en el plano

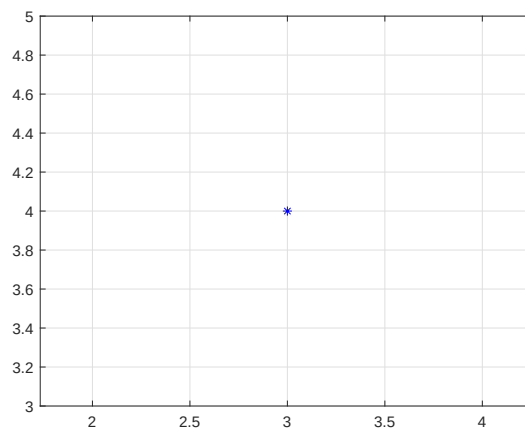
Es interesante ver la representación gráfica de los números complejos en el plano. Para ello se pueden utilizar varias funciones como `plot`, `ezplot` o `compass`. Estas funciones tienen diferentes tratamientos.

Ejemplo 3.3 Representar el número $c = 3 + 4i$. □

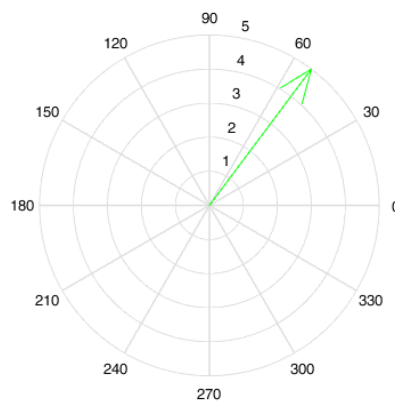
$$c = 3 + 4i$$

$$c = 3.0000 + 4.0000i$$

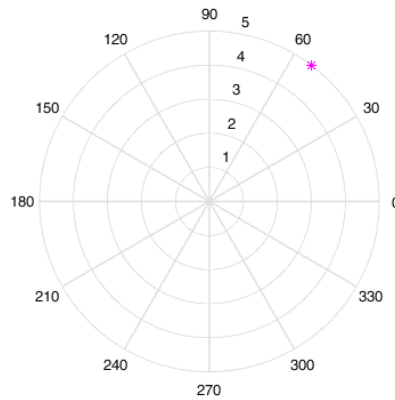
```
plot(c, 'b*')
grid on
axis equal
```



```
figure
compass(c, 'g')
```



```
figure
polar( angle(c), abs(c), 'm*' )
```



La primera representación es la correspondiente a los ejes cartesianos.

Las otras dos están referidas a los ejes polares mediante módulo y argumento.

Ejemplo 3.4 Representar los complejos $c = 3 + 4i$, $d = 1 - 2i$ y mediante la regla del paralelogramo $c + d$. □

$$c = 3 + 4i$$

$$c = 3.0000 + 4.0000i$$

$$d = 1 - 2i$$

$$d = 1.0000 - 2.0000i$$

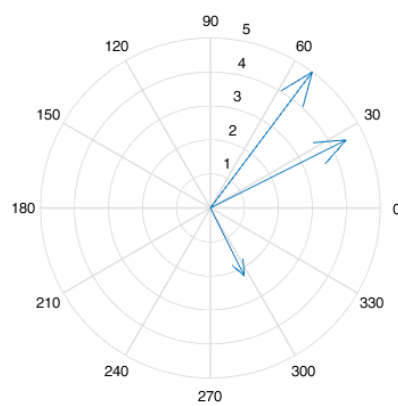
$$s = c + d$$

$$s = 4.0000 + 2.0000i$$

$$ZZ = [c \ d \ s]$$

$$ZZ = \begin{matrix} 1 \times 3 \text{ complex} \\ 3.0000 + 4.0000i & 1.0000 - 2.0000i & 4.0000 + 2.0000i \end{matrix}$$

$$\text{compass}(ZZ)$$



Parte III

Sistemas de Ecuaciones Lineales

Capítulo 4

Sistemas de ecuaciones lineales

En el proceso para la resolución de sistemas de ecuaciones lineales se contemplan dos etapas fundamentales: clasificación y resolución propiamente dicha.

4.0.1. Clasificación de sistemas

En esta fase del proceso de resolución es cuando se discute si el sistema es resoluble, es decir si es compatible o no, y, en el caso de serlo, cuántas soluciones tiene, es decir, si el sistema es determinado o no.

Para esta discusión son útiles las funciones de MATLAB:

Función	Salida
det(A)	Determinante de la matriz A
rank(A)	Rango de la matriz A
rref(A)	Matriz escalonada reducida de la matriz A

Veamos ahora cómo podemos abordar el problema del estudio de la compatibilidad.

Estudio mediante el rango

El rango de las matrices de coeficientes y ampliada nos aportan la información necesaria para determinar el tipo de sistema que se estudia.

Sistemas compatibles Se dice que un sistema es compatible cuando admite solución.

Sistemas compatibles determinados Estos sistemas son aquellos que admiten una única solución. El rango de la matriz de coeficientes coincide con el número de incógnitas con lo cuál, nos basta comprobar que el determinante de dicha matriz es no nulo.

Ejemplo 4.1 Consideremos el sistema siguiente:

$$\left. \begin{array}{rrcr} x & + & y & + & z & = & 6 \\ x & & & - & 2z & = & 4 \\ & & y & + & z & = & 2 \end{array} \right\} \quad (4.0.1)$$

La matriz A de coeficientes y el vector b de términos independientes del sistema son:

$$A = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 0 & -2 \\ 0 & 1 & 1 \end{pmatrix} \quad b = \begin{pmatrix} 6 \\ 4 \\ 2 \end{pmatrix}$$

Las introducimos con MATLAB:

```
>> A=[1,1,1;1,0,-2;0,1,1],b=[6 4 2]'
```

Se construye la matriz ampliada:

```
>> Ab=[A b]
```

Hacemos el estudio de los rangos de la matriz de coeficientes y de la matriz ampliada comparándolos:

```
>> rank(Ab)==rank(A)
```

```
>> rank(A)==3
```

y, como vemos, dado que coinciden los rangos con el número de incógnitas, el sistema es compatible determinado y tendrá una única solución.

Aunque, como comentábamos, basta obtener el determinante de la matriz y comprobar que es no nulo para ver que es compatible determinado.

```
>> det(A)
```

□

Sistemas compatibles indeterminados Estos sistemas son aquellos que admiten infinitas soluciones. El rango de la matriz de coeficientes del sistema debe coincidir con el de la matriz ampliada y, además, ser menor que el número de incógnitas.

Ejemplo 4.2 Consideremos el sistema siguiente:

$$\left. \begin{array}{rrcr} x & + & y & + & z & = & 6 \\ x & & & - & 2z & = & 4 \\ 2x & + & y & - & z & = & 10 \end{array} \right\} \quad (4.0.2)$$

La matriz A de coeficientes y el vector b de términos independientes del sistema son:

$$A = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 0 & -2 \\ 2 & 1 & -1 \end{pmatrix} \quad b = \begin{pmatrix} 6 \\ 4 \\ 10 \end{pmatrix}$$

Las introducimos con MATLAB:

```
>> A=[1,1,1;1,0,-2;2,1,-1],b=[6 4 10]'
```

Se construye la matriz ampliada:

```
>> Ab=[A b]
```

Hacemos el estudio de los rangos de la matriz de coeficientes y de la matriz ampliada comparándolos:

```
>> rank(Ab)==rank(A)
>> rank(A)==3
```

y, como vemos, dado que, aunque coinciden los rangos, pero no con el número de incógnitas, el sistema es compatible indeterminado y tendrá infinitas soluciones. $\square$

Sistemas incompatibles Son aquellos sistemas que no tienen solución.

Ejemplo 4.3 Consideremos el sistema siguiente:

$$\left. \begin{array}{rrcr} x & + & y & + & z & = & -6 \\ x & & & - & 2z & = & 4 \\ 2x & + & y & - & z & = & 18 \end{array} \right\} \quad (4.0.3)$$

La matriz A de coeficientes y el vector b de términos independientes del sistema son:

$$A = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 0 & -2 \\ 2 & 1 & -1 \end{pmatrix} \quad b = \begin{pmatrix} -6 \\ 4 \\ 18 \end{pmatrix}$$

Las introducimos con MATLAB:

```
>> A=[1,1,1;1,0,-2;2,1,-1],b=[-6 4 18]'
```

Se construye la matriz ampliada:

```
>> Ab=[A b]
```

Hacemos el estudio de los rangos de la matriz de coeficientes y de la matriz ampliada comparándolos:

```
>> rank(Ab)==rank(A)
```

y, como vemos, dado que no coinciden los rangos, el sistema es incompatible y no tendrá solución. $\square$

Estudio mediante el método de Gauss

La orden de MATLAB que nos permite realizar la clasificación mediante el método de Gauss es *rref*.

En primer lugar, recordemos algunas definiciones.

- El primer elemento no nulo en cada fila de una matriz se denomina *pivote*.
- Una matriz se dice que está en forma escalonada por filas si:
 - las filas de ceros aparecen en la parte inferior de la matriz
 - cada pivote es 1
 - cada pivote aparece en una columna estrictamente a la derecha del pivote de la fila anterior
- Se dice que una matriz está en forma reducida por filas si es escalonada y satisface, además, que cada pivote es el único elemento no nulo en su columna.

Se sabe que toda matriz es equivalente a una matriz reducida por filas, es decir, que mediante transformaciones elementales (por filas) toda matriz se puede convertir en una matriz reducida por filas.

Veamos, ahora, la clasificación de los sistemas anteriores mediante la orden *rref*.

Para el caso de los sistemas compatibles se tiene:

Ejemplo 4.4 Consideremos el sistema del ejemplo 4.1.

Al realizar

```
>> rref(Ab)
```

ans =

1	0	0	4
0	1	0	2
0	0	1	0

observamos que la matriz resultante es la identidad ampliada con una columna por lo que, obviamente, el rango de ambas matrices coincide y también lo hace con el número de incógnitas y el sistema es, por tanto compatible determinado. $\square$

Ejemplo 4.5 Consideremos el sistema del ejemplo 4.2.

Al realizar

```
>> rref(Ab)
```

ans =

1	0	-2	4
0	1	3	2
0	0	0	0

observamos que la última fila de la matriz resultante es nula por lo que, obviamente, el rango de la matriz de coeficientes y de la ampliada son iguales pero son menores que el número de incógnitas y el sistema es, por tanto compatible pero indeterminado. $\square$

Ahora para los sistemas incompatibles tenemos:

Ejemplo 4.6 Consideremos el sistema del ejemplo 4.3.

Al realizar

```
>> rref(Ab)
```

ans =

1	0	-2	0
0	1	3	0
0	0	0	1

observamos que la matriz resultante tiene la última fila nula salvo la última componente por lo que, obviamente, el rango de ambas matrices no coincide y el sistema es, por tanto, incompatible.

Además, la última fila se corresponde a la ecuación $0 \cdot x + 0 \cdot y + 0 \cdot z = 1$ que no tiene solución y por lo tanto tampoco el sistema. $\square$

Nota 4.1 Es importante reseñar que con la orden `rref` podemos, de la misma manera que se puede realizar "en papel", obtener la inversa de una matriz.

Ejemplo 4.7 Obtener la inversa de la matriz

$$A = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 0 & -2 \\ 0 & 1 & 1 \end{pmatrix}$$

Veamos cómo se puede hacer con MATLAB.

En primer lugar, introducimos la matriz

```
>> A=[1,1,1;1,0,-2;0,1,1]
```

A =

1	1	1
1	0	-2
0	1	1

Ampliamos la matriz A con la identidad y aplicamos la orden `rref`

```
>> AI=[A eye(size(A))]
```

AI =

1	1	1	1	0	0
1	0	-2	0	1	0
0	1	1	0	0	1

```
>> B=rref(AI)
```

B =

1	0	0	1	0	-1
0	1	0	-1/2	1/2	3/2
0	0	1	1/2	-1/2	-1/2

Ahora, nos quedamos con la parte ampliada de la matriz que es la inversa de la matriz A.

```
>> AINV(1:3,1:3)=B(1:3,4:6)
```

AINV =

$$\begin{array}{ccc} 1 & 0 & -1 \\ -1/2 & 1/2 & 3/2 \\ 1/2 & -1/2 & -1/2 \end{array}$$

Finalmente, podemos comprobar que, en efecto, lo es.

```
>> A*AINV
```

```
ans =
```

$$\begin{array}{ccc} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{array}$$

□

4.0.2. Resolución de sistemas

Vayamos, ahora, a la resolución de los sistemas de ecuaciones.

Para la resolución, además de la orden `rref`, es importante:

Función	Salida
<code>solve(f1,...,fn,x1,...,xn,'ReturnConditions',true)</code>	Calcula las soluciones del sistema de ecuaciones lineales $f1 = 0, \dots, fn = 0$ e incógnitas $x1, \dots, xn$ siendo $f1, \dots, fn$ expresiones simbólicas conteniendo las variables simbólicas $x1, \dots, xn$.

'ReturnConditions', true provoca que se den las condiciones sobre la solución las condiciones sobre la solución, por ejemplo:

```
syms x y real
[S, params, conditions] = solve(x ^ (1/2) == y, x, ...
    'ReturnConditions', true)
```

```
S =
    y2
params =
Empty sym: 1-by-0
conditions =
    0 ≤ y
```

lo que indica que, necesariamente y debe ser positiva.

Sistemas compatibles

Vamos a ver cómo obtener las soluciones de los sistemas compatibles que, como ya hemos comentado, pueden ser infinitas o sólo una.

Sistemas compatibles determinados Veamos cómo obtener la solución única de estos sistemas.

Ejemplo 4.8 Siguiendo con el ejemplo 4.4, la matriz resultante de aplicar rref a la matriz ampliada representa un sistema equivalente al inicial 4.0.1

$$\left. \begin{array}{rcl} x & = & 4 \\ y & = & 2 \\ z & = & 0 \end{array} \right\} \quad (4.0.4)$$

cuya única solución es (4,2,0) y por tanto ésta es la única solución del sistema 4.0.1.

Por otro lado, si conocemos la naturaleza del sistema, para obtener la solución podemos multiplicar ambos miembros del sistema en forma matricial por la inversa de la matriz de coeficientes, es decir despejamos x , ya que en este caso es una matriz no singular.

$$A \cdot x = b \implies x = A^{-1} \cdot b$$

Así, dada la matriz A de coeficientes y el vector b de términos independientes del sistema:

```
>> A
```

```
A =
```

```

1      1      1
1      0     -2
0      1      1
```

```
>> InvA=A^-1
```

```
InvA =
```

```

1      0     -1
-1/2   1/2   3/2
1/2   -1/2  -1/2
```

```
>> x=InvA*b
```

```
x =
```

```

4
2
0
```

□

Sistemas compatibles indeterminados Ahora estamos ante sistemas que tienen infinitas soluciones y vamos a ver cómo se obtienen todas ellas.

Ejemplo 4.9 Siguiendo con el ejemplo 4.5, al aplicar la orden rref a la matriz ampliada del sistema obtuvimos una cuya última fila era nula y, además, sólo tenía dos pivotes que estaban en las columnas 1 y 2.

Como la última fila es nula, podríamos considerar que el sistema 4.5 es equivalente al sistema

$$\left. \begin{array}{rcl} x & - & 2z = 4 \\ y & + & 3z = 2 \end{array} \right\} \quad (4.0.5)$$

Ahora, resolvemos cada ecuación para su variable pivote en función de las variables libres

$$\left. \begin{array}{rcl} x & = & 4 + 2z \\ y & = & 2 - 3z \end{array} \right\} \quad (4.0.6)$$

Se suelen tomar parámetros para representar las variables libres, por ejemplo, en nuestro caso, tomando $z = \lambda$, el sistema tiene infinitas soluciones descritas como

$$x = 4 + 2\lambda, \quad y = 2 - 3\lambda, \quad z = \lambda \quad \lambda \in \mathbb{R}$$

Por ejemplo, para $\lambda = 0$ se obtiene la solución $(4, 2, 0)$ o, para $\lambda = 2$, obtenemos la solución $(8, -4, 2)$. $\square$

Sistemas incompatibles

Abordar el problema de los sistemas incompatibles es trivial ya que, como comentábamos en el ejemplo 4.6, al aplicar la orden `rref` a la matriz ampliada, obteníamos la ecuación $0 \cdot x + 0 \cdot y + 0 \cdot z = 1$ asociada a la última fila que no tiene solución y por lo tanto tampoco el sistema.

La orden `solve`

En primer lugar, debemos tener en cuenta que tiene una sintaxis que nos permite trabajar directamente con las ecuaciones de los sistemas ya que es un operador simbólico. También, permite resolver respecto de las variables simbólicas que nosotros elijamos.

La solución aportada por la orden `solve` depende, obviamente, de la naturaleza del sistema.

Veamos algunos ejemplos para observar las diferencias.

Ejemplo 4.10 Resuélvase el siguiente sistema de ecuaciones lineales compatible determinado

$$\left\{ \begin{array}{rcl} a + b + c & = & 2 \\ a - b - c & = & 0 \\ a + b - c & = & 4 \end{array} \right. \quad (4.0.7)$$

Utilizaremos la orden `solve` de MATLAB después de haber definido las variables simbólicas, almacenaremos la solución en un vector simbólico y posteriormente, visualizaremos la solución. $\square$

```
syms a b c
sol = solve(a + b + c == 2, a - b - c == 0, a + b - c == 4, 'ReturnConditions', ...
    true)
```

```
sol =

    a: [1x1 sym]
    b: [1x1 sym]
    c: [1x1 sym]
parameters: [1x0 sym]
conditions: [1x1 sym]
```

```
s = [ sol.a sol.b sol.c ]
```

```
s =
( 1 2 -1 )
```

Ejemplo 4.11 *Resuélvase el siguiente sistema de ecuaciones lineales*

$$\begin{cases} 5x + 5y - 2z - 2t = 6 \\ 5x - 10y + 7z + 7t = 9 \\ 5x + z + t = 7 \\ +5y - 3z - 3t = -1 \end{cases} \quad (4.0.8)$$

Seguiremos los mismos pasos que en caso anterior □

```
syms x y z t
sol = solve(5*x + 5*y - 2*z - 2*t == 6, 5*x - 10*y + 7*z + 7*t == 9, ...
    5*x + z + t == 7, 5*y - 3*z - 3*t == -1, 'ReturnConditions', true)
```

```
sol =
      t: [1 x1 sym]
      x: [1 x1 sym]
      y: [1 x1 sym]
      z: [1 x1 sym]
parameters: [1 x2 sym]
conditions: [1 x1 sym]
```

```
s = [ sol.x sol.y sol.z sol.t ]
```

```
s =
( 4/3 - z2/3 z2 z1 5z2/3 - z1 + 1/3 )
```

```
sol.parameters
```

```
ans =
( z1 z2 )
```

```
sol.conditions
```

```
ans =
TRUE
```

Finalmente, veamos un sistema incompatible.

Ejemplo 4.12 *Resuélvase el siguiente sistema de ecuaciones lineales en el que la primera y la última ecuación son incompatibles.*

$$\begin{cases} a + b + c = 2 \\ a - b - c = 0 \\ a + b + c = 4 \end{cases} \quad (4.0.9)$$

```
syms a b c
sol = solve(a + b + c == 2, a - b - c == 0, a + b + c == 4, ...
    'ReturnConditions', true)
```

```
sol =
      a: [0x1 sym]
      b: [0x1 sym]
      c: [0x1 sym]
parameters: [1x0 sym]
conditions: [0x1 sym]
```

Observamos que la orden devuelve todas las variables vacías. □

La orden *solve* permite elegir las variables y trabajar con parámetros.

Ejemplo 4.13 La siguiente sintaxis devuelve la solución del sistema en las variables u y v , considerando que a es un parámetro. □

```
syms a u v
sol = solve(a*u ^ 2 + v ^ 2 == 0, u - v == 1, 'ReturnConditions', true)
```

```
sol =
      u: [3x1 sym]
      v: [3x1 sym]
parameters: [1x0 sym]
conditions: [3x1 sym]
```

```
s = [sol.u sol.v]
```

```
s =
      (
      1 - \frac{1}{2} \frac{a + \sqrt{-a}}{a + 1} \quad -\frac{1}{2} \frac{a + \sqrt{-a}}{a + 1}
      1 - \frac{a - \sqrt{-a}}{a + 1} \quad -\frac{a - \sqrt{-a}}{a + 1}
      )
```

Ejemplo 4.14 La siguiente sintaxis devuelve la solución del sistema en las variables a y u tomando v como parámetro. □

```
syms a u v
sol = solve(a*u ^ 2 + v ^ 2 == 0, u - v == 1, a, u, 'ReturnConditions', true)
```

```
sol =
      a: [1x1 sym]
      u: [1x1 sym]
parameters: [1x0 sym]
conditions: [1x1 sym]
```

```
s = [sol.a sol.u]
```

$$s = \begin{pmatrix} -\frac{v^2}{v^2+2v+1} & v+1 \end{pmatrix}$$

```
syms a u v
sol = solve(a*u ^ 2 + v ^ 2 == 0,u-v == 1,a,u,'ReturnConditions',true)
```

$$\text{sol} = \begin{array}{ll} a: & [1 \times 1 \text{ sym}] \\ u: & [1 \times 1 \text{ sym}] \\ \text{parameters:} & [1 \times 0 \text{ sym}] \\ \text{conditions:} & [1 \times 1 \text{ sym}] \end{array}$$

```
s = [sol.a sol.u]
```

$$s = \begin{pmatrix} -\frac{v^2}{v^2+2v+1} & v+1 \end{pmatrix}$$

Ejemplo 4.15 Hállense u y v , tales que se verifiquen las ecuaciones $u + v = 2x$, $u - v = 2y$. $\square$

Solución:

```
clear all
syms u v x y
f1 = u + v - 2*x
```

$$f1 = u + v - 2x$$

```
f2 = u - v - 2*y
```

$$f2 = u - v - 2y$$

```
sol = solve(f1 == 0,f2 == 0,u,v,'ReturnConditions',true)
```

$$\text{sol} = \begin{array}{ll} u: & [1 \times 1 \text{ sym}] \\ v: & [1 \times 1 \text{ sym}] \\ \text{parameters:} & [1 \times 0 \text{ sym}] \\ \text{conditions:} & [1 \times 1 \text{ sym}] \end{array}$$

```
s = [sol.u sol.v]
```

$$s = \begin{pmatrix} x+y & x-y \end{pmatrix}$$

Parte IV

Espacios Vectoriales

Capítulo 5

Espacios vectoriales

Comenzaremos este apartado resolviendo problemas relacionados con sistemas de vectores en espacios vectoriales. Estudiaremos la dependencia lineal de un conjunto de vectores, calcularemos la dimensión del subespacio que engendra, como obtener una base del mismo, etc.

Seguidamente, abordaremos problemas asociados a subespacios vectoriales: obtención de los subespacios suma e intersección, suma directa, cálculo de bases, etc.

Concluiremos el tema estudiando los cambios de base.

5.1. Dependencia e independencia lineal

La orden necesaria para ver si unos vectores son, o no, linealmente independientes es, obviamente, la que calcula el rango de una matriz: `rank`.

Además, podemos obtener los coeficientes necesarios de la combinación lineal de unos vectores para obtener otro. Esto se realiza con la orden `solve`.

Ejemplo 5.1 Compruébese si son linealmente dependientes los vectores:

$$\vec{v}_1 = (1, 3, 2, -3), \vec{v}_2 = (1, -1, 0, 2), \vec{v}_3 = (-2, -1, -3, 0)$$

□

```
S = [1 3 2 -3; 1 -1 0 2; -2 -1 -3 0]
```

```
S = 3x4
     1     3     2    -3
     1    -1     0     2
    -2    -1    -3     0
```

```
rank(S)
```

```
ans = 3
```

Como el rango es 3 podemos afirmar que los vectores son linealmente independientes.

```
rref(S)
```

```
ans = 3x4
    1.0000         0         0    0.8333
         0    1.0000         0   -1.1667
         0         0    1.0000   -0.1667
```

También viendo el sistema equivalente escalonado, como no tiene filas nulas, los vectores son linealmente dependientes.

Ejemplo 5.2 *Lo mismo para los vectores:*

$$\vec{v}_1 = (1, 3, 2, -3), \vec{v}_2 = (1, -1, 0, 2), \vec{v}_3 = (-2, -1, -3, 0), \vec{v}_4 = (0, 1, -1, -1)$$

En caso de ser linealmente dependientes, escribir la combinación de los tres primeros que obtiene el último □

```
S = [1 3 2 -3; 1 -1 0 2 ; -2 -1 -3 0; 0 1 -1 -1]
```

```
S = 4x4
     1     3     2    -3
     1    -1     0     2
    -2    -1    -3     0
     0     1    -1    -1
```

```
rank(S)
```

```
ans = 3
```

Observamos que el rango es 3, por tanto uno de ellos es combinación lineal de los otros.

Aplicamos la orden *rref* a la matriz formada por los vectores por columnas.

```
rref(S.')
```

```
ans = 4x4
     1     0     0     1
     0     1     0     1
     0     0     1     1
     0     0     0     0
```

Con lo que obtenemos en la última columna la combinación de los tres primeros vectores que dan lugar al último.

```
S(1,:) + S(2,:) + S(3,:)
```

```
ans = 1x4
     0     1    -1    -1
```

5.2. Base y dimensión

Cuando se tiene un subespacio vectorial definido a partir de una familia de vectores resulta de especial importancia conocer su dimensión y una base. Para esto utilizaremos las órdenes *rank* para su dimensión y la orden *rref* para obtener una base.

Ejemplo 5.3 Obtener la dimensión y una base del espacio vectorial generado por la familia de vectores $B = \{\vec{v}_1 = (1, 3, 2, -3), \vec{v}_2 = (1, -1, 0, 2), \vec{v}_3 = (-2, -1, -3, 0), \vec{v}_4 = (0, 1, -1, -1)\}$

En el ejemplo anterior 5.2 hemos visto que el rango de la matriz generada por las filas de los vectores de B es 3 que es, por tanto, la dimensión del espacio generado por B .

Para obtener una base podemos realizar:

```
>> rref([1,3,2,-3;1,-1,0,2;-2,-1,-3,0;0,1,-1,-1])
```

ans =

1	0	0	5/6
0	1	0	-7/6
0	0	1	-1/6
0	0	0	0

con lo que se consigue una base del subespacio $\langle B \rangle$

$$\mathcal{B} = \{(1, 0, 0, 5/6), (0, 1, 0, -7/6), (0, 0, 1, -1/6)\}$$

□

5.3. Subespacios vectoriales. Suma e intersección de subespacios

Veamos la generación de subespacios a partir de sistemas de vectores. Con los ejemplos siguientes vamos a ilustrar la construcción de los subespacios suma e intersección de otros dados.

Ejemplo 5.4 En $\mathbb{R}^4$ se consideran los subespacios $S = \mathbb{R} \langle (1, 1, 0, 0), (0, 0, 1, 0), (0, 0, 0, 1) \rangle$ y $T = \mathbb{R} \langle (1, 1, 2, 1), (2, 0, -1, 1) \rangle$. Hallar bases y dimensiones de los subespacios S , T , $S+T$ y $S \cap T$.

En primer lugar, veamoslo para S y T .

```
>> S=[1, 1, 0, 0;0, 0, 1, 0;0, 0, 0, 1]
```

S =

1	1	0	0
0	0	1	0
0	0	0	1

```
>> T=[1, 1, 2, 1;2, 0,-1, 1]
```

T =

1	1	2	1
2	0	-1	1

```
>> rS=rank(S),rT=rank(T)
```

rS =

3

rT =

2

Como los rangos coinciden con el número de vectores que los generan, conforman una base. Aunque también podríamos obtener una base de ambos subespacios con la orden rref.

```
>> format rat
```

```
>> Base_S=rref(S(1:rS,:)) % Base de <S>
```

Base_S =

1	1	0	0
0	0	1	0
0	0	0	1

```
>> Base_T=rref(T(1:rT,:)) % Base de <T>
```

Base_T =

1	0	1/2	1/2
0	1	3/2	1/2

Para el caso de $S+T$, se estudia el rango de la matriz formada por los vectores generadores de S y de T , o de la base de ambos subespacios.

```
>> rST=rank([S;T])
```

rST =

4

y, por tanto, como la dimensión es la misma que la del espacio total $\mathbb{R}^4$ coinciden y una base de $S+T$ es, obviamente, la base canónica, base que obtenemos con la orden rref.

```
>> baseST=rref([S;T])
```

ans =

1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1
0	0	0	0

Finalmente, para S T , en primer lugar, vamos a obtener las ecuaciones de cada subespacio. Para ello:

```
>> ecuacionesS=null(S,'r') %la opción r consigue las ecuaciones
con coeficientes 'racionales'
```

```
ecuacionesS =
```

```

-1
 1
 0
 0
```

```
>> ecuacionesT=null(T,'r')
```

```
ecuacionesT =
```

```

 1/2      -1/2
-5/2      -1/2
 1         0
 0         1
```

lo que nos proporciona las ecuaciones de S por un lado $-x + y = 0$ y de T por otro $\frac{1}{2}x - \frac{5}{2}y + z = 0$ y $-\frac{1}{2}x - \frac{1}{2}y + t = 0$.

Ahora, los vectores que verifican las 3 ecuaciones son los de $S \cap T$

```
>> sol=solve(-x+y,(1/2)*x-(5/2)*y+z,-(1/2)*x-(1/2)*y+t,'x,y,z,t')
```

```
Warning: 3 equations in 4 variables. New variables might be
introduced.
```

```
sol =
```

```

t: [1x1 sym]
x: [1x1 sym]
y: [1x1 sym]
z: [1x1 sym]
```

```
>> sol.x,sol.y,sol.z,sol.t
```

```
ans =
```

```
z1/2
```

```
ans =
```

```
z1/2
```

```
ans =
```

z1

ans =

z1/2

esto es, $S \cap T = \langle (1/2, 1/2, 1, 1/2) \rangle$

□

Ejemplo 5.5 Obtener el espacio suma de $V_1 + V_2$ donde

$$V_1 = \langle (1, 2, 5, 3, 2), (3, 1, 5, -6, 6), (1, 1, 3, 2, 0) \rangle$$

$$V_2 = \langle (2, 1, 4, -3, 4), (3, 1, 3, -2, 2), (9, 2, 3, -1, -2) \rangle$$

¿Es suma directa de los dos subespacios?

En primer lugar, introducimos las matrices de los sistemas de vectores y calculamos el rango y una base del subespacio suma.

```
>> V1 = [1 2 5 3 2; 3 1 5 -6 6; 1 1 3 2 0], V2 = [2 1 4 -3 4; 3 1 3 -2 2;
9 2 3 -1 -2]
```

V1 =

1	2	5	3	2
3	1	5	-6	6
1	1	3	2	0

V2 =

2	1	4	-3	4
3	1	3	-2	2
9	2	3	-1	-2

```
>> rank([V1;V2])
```

ans =

4

```
>> rref([V1;V2])
```

ans =

1	0	0	0	-1
0	1	0	0	3
0	0	1	0	0
0	0	0	1	-1
0	0	0	0	0
0	0	0	0	0

Con la última orden hemos obtenido una matriz de 6 filas (dos de ellas nulas) que conforman una base (salvo las nulas) del subespacio suma.

Ahora calculamos los rangos de V_1 y V_2 y comparamos su suma con la dimensión del subespacio suma.

```
>> rank(V1),rank(V2)
```

```
ans =
```

```
3
```

```
ans =
```

```
2
```

```
>> rank(V1)+rank(V2)==rank([V1;V2])
```

```
ans =
```

```
0
```

Con esto hemos comprobado que no es suma directa ya que la suma de dimensiones de los subespacios no es igual a la dimensión del subespacio suma. $\square$

5.4. Cambio de base

Sean $\mathcal{B} = \{u_1, \dots, u_n\}$ y $\mathcal{B}' = \{v_1, \dots, v_n\}$ dos bases de un espacio vectorial V de dimensión n . Supongamos que un vector $v \in V$ tiene unas coordenadas $X = (x_1, \dots, x_n)$ en la base $\mathcal{B}$ y unas coordenadas $Y = (y_1, \dots, y_n)$ en la base $\mathcal{B}'$, es decir, que

$$v = x_1 u_1 + \dots + x_n u_n = y_1 v_1 + \dots + y_n v_n$$

Supongamos que conocemos las coordenadas de los vectores de la base (nueva), $\mathcal{B}'$, respecto de la base (antigua), $\mathcal{B}$, es decir, que sabemos que

$$\left. \begin{aligned} v_1 &= p_{11}u_1 + \dots + p_{n1}u_n \\ v_2 &= p_{12}u_1 + \dots + p_{n2}u_n \\ &\dots\dots\dots \\ v_n &= p_{1n}u_1 + \dots + p_{nn}u_n \end{aligned} \right\}$$

Según se ha visto en teoría, la matriz

$$P = \begin{pmatrix} p_{11} & p_{12} & \dots & p_{1n} \\ p_{21} & p_{22} & \dots & p_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ p_{n1} & p_{n2} & \dots & p_{nn} \end{pmatrix}$$

es la matriz de paso o del cambio de base de la base (antigua) $\mathcal{B}$ a la base (nueva) $\mathcal{B}'$, y la relación entre las coordenadas X e Y de v es: $X = PY$.

Ejemplo 5.6

1. Un vector $x \in \mathbb{R}^3$ tiene por coordenadas $(1,2,3)$ respecto de la base $B = \{v_1, v_2, v_3\}$. Obtener las coordenadas de x en la base $B' = \{u_1, u_2, u_3\}$ sabiendo que: $u_1 = 3v_1 + 2v_2 - v_3$, $u_2 = 4v_1 + v_2 + v_3$ y $u_3 = 2v_1 - v_2 + v_3$
2. Las coordenadas de un vector x respecto de una base

$$B = \{u_1 = (1,0,2,1), u_2 = (0,3,2,1), u_3 = (3,4,1,0), u_4 = (1,1,0,1)\}$$

son $(3,1,2,6)$. Obténganse sus coordenadas respecto de la base canónica.

1. Sabemos que la matriz de cambio de base es:

$$\begin{pmatrix} 3 & 4 & 2 \\ 2 & 1 & -1 \\ -1 & 1 & 1 \end{pmatrix}$$

luego, para obtener las coordenadas nuevas debemos multiplicar las viejas por la matriz inversa del cambio.

```
>> P=[3,4,2;2,1,-1;-1,1,1]
```

```
P =
```

```

     3         4         2
     2         1        -1
    -1         1         1
```

```
>> x=[1 2 3]%coordenadas viejas
```

```
x =
```

```

     1         2         3
```

```
>> y=P^-1*x'%coordenadas nuevas
```

```
y =
```

```

   -5/2
   15/4
  -13/4
```

2. Dado que las coordenadas de los vectores de la base B están referidas a la base canónica, sabemos que la matriz del cambio es:

$$P = \begin{pmatrix} 1 & 0 & 3 & 1 \\ 0 & 3 & 4 & 1 \\ 2 & 2 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{pmatrix}$$

En MATLAB, podemos introducir las coordenadas en una matriz y trasponerla y como las coordenadas nuevas ahora son las que tenemos respecto de la base B , multiplicaremos éstas por la matriz

```
>> P=[1,0,2,1;0,3,2,1;3,4,1,0;1,1,0,1]'
```

```
P =
```

```

1         0         3         1
0         3         4         1
2         2         1         0
1         1         0         1
```

```
>> P*[3 1 2 6]'
```

```
ans =
```

```

15
17
10
10
```

□

Ejemplo 5.7 Se consideran los vectores de $\mathbb{R}^3$:

$$v_1 = (-2, 2, 3), v_2 = (-8, 5, 2), v_3 = (-7, 2, 6)$$

y la matriz

$$\begin{pmatrix} 1 & 2 & -1 \\ -3 & -5 & 0 \\ 4 & 6 & 1 \end{pmatrix}$$

1. Encuéntrese una base $\mathcal{B}_1$ tal que P sea la matriz de cambio de base de $\mathcal{B}_1$ a $\mathcal{B}_2 = \{v_1, v_2, v_3\}$.
2. Calcúlense las coordenadas, en la base $\mathcal{B}_2$ del vector v cuyas coordenadas en la primera base $\mathcal{B}_1$ son $(1, 2, 3)$.

1. Sean B_1 la matriz de coeficientes de los vectores de la base $\mathcal{B}_1$ (precisamente es la que debemos calcular) y B_2 la matriz de coeficientes de los vectores de la base $\mathcal{B}_2$.

Dado un vector $\vec{v}$, consideremos x_c sus coordenadas respecto de la base canónica, x_1 sus coordenadas respecto de la base $\mathcal{B}_1$ y x_2 sus coordenadas respecto de la base $\mathcal{B}_2$. Así, se tiene:

$$x_c = B_1 \cdot x_1 = B_2 \cdot x_2$$

y además $x_1 = P \cdot x_2$, luego

$$B_1 = B_2 \cdot P^{-1}$$

```
>> B2=[-2,2,3;-8,5,2;-7,2,6]'
```

```
B2 =
```

```

-2         -8        -7
 2          5         2
 3          2         6
```

```
>> P=[ 1  2  -1 ; -3  -5  0; 4  6  1 ]
```

P =

1	2	-1
-3	-5	0
4	6	1

>> B1=B2*P^-1

B1 =

28	38	21
-9	-13	-7
-3	2	3

Luego la base buscada es la formada por los vectores:

>> u1=B1(:,1),u2=B1(:,2),u3=B1(:,3)

u1 =

28
-9
-3

u2 =

38
-13
2

u3 =

21
-7
3

2. Para obtener las coordenadas pedidas:

>> x2=P^-1*[1 2 3]'

x2 =

36
-22
-9

□

Parte V

Aplicaciones Lineales y Diagonalización

Capítulo 6

Aplicaciones lineales

En este capítulo resolveremos, con ayuda del MatLab, ejercicios que nos permitan adquirir destreza en el manejo de las aplicaciones lineales, gracias a la posibilidad de representarlas por medio de matrices.

Comenzaremos recordando como se representa matricialmente una aplicación lineal, para, a partir de dicha representación obtener bases de los conjuntos núcleo e imagen. Asimismo, veremos como afectan a esta representación los cambios de base que se realizan en los espacios vectoriales inicial y final.

6.1. Representación de una aplicación lineal

Una aplicación f entre los espacios vectoriales U y V , definidos sobre un cuerpo K , de dimensión n y m respectivamente,

$$f: U \longrightarrow V$$

es lineal si se cumple

$$f(tx + sy) = tf(x) + sf(y) \quad \forall t, s \in K, \forall x, y \in V$$

Fijadas unas bases $\mathcal{B}_U = \{u_i, i = 1, \dots, n\}$ y $\mathcal{B}_V = \{v_j, j = 1, \dots, m\}$ de los espacios vectoriales U y V , respectivamente, a toda aplicación lineal f se le puede asociar una matriz A . Dicha matriz A tiene por columnas las coordenadas de los vectores $f(u_i)$ respecto de la base $\mathcal{B}_V$, por lo que $A \in MK(m, n)$.

La ecuación matricial de una aplicación lineal es $Y = AX$, siendo X la matriz columna coordenada de un vector $x \in U$ respecto de una base $\mathcal{B}_U = \{u_i, i = 1, 2, \dots, n\}$ e Y la matriz columna coordenada del vector $y = f(x) \in V$ respecto de una base $\mathcal{B}_V = \{v_j, j = 1, 2, \dots, m\}$.

Ejemplo 6.1 Sea una aplicación $f: \mathbb{R}^3 \longrightarrow \mathbb{R}^2$ definida por

$$f(x, y, z) = (2x + z, 3x - y)$$

1. Pruébese que f es lineal.
2. Hállese su ecuación matricial respecto de las bases canónicas.
1. Veamos que se verifica que $f(tu + sv) = tf(u) + sf(v)$ cualesquiera que sean $t, s \in \mathbb{R}$ y $u, v \in \mathbb{R}^3$, previa definición de la aplicación lineal

```
syms x y z a b r s t
f(x, y, z) = [2*x + z 3*x - y]
```

$$f(x, y, z) = \begin{pmatrix} 2x+z & 3x-y \end{pmatrix}$$

```
simplify ( f ( a*x + b*r , a*y + b*s , a*z + b*t ) - ( a*f ( x , y , z ) + b*f ( r , s , t ) ) )
```

$$ans = \begin{pmatrix} 0 & 0 \end{pmatrix}$$

2. Para obtener la matriz asociada a la aplicación lineal respecto de las bases canónicas debemos buscar las imágenes de la base canónica de $\mathbb{R}^3$ expresadas en la base canónica de $\mathbb{R}^2$ y escribirlas por columnas en una matriz.

$$v1 = f(1, 0, 0), \quad v2 = f(0, 1, 0), \quad v3 = f(0, 0, 1)$$

$$\begin{aligned} v1 &= \begin{pmatrix} 2 & 3 \end{pmatrix} \\ v2 &= \begin{pmatrix} 0 & -1 \end{pmatrix} \\ v3 &= \begin{pmatrix} 1 & 0 \end{pmatrix} \end{aligned}$$

$$A = [v1, \quad v2, \quad v3]$$

$$A = \begin{pmatrix} 2 & 0 & 1 \\ 3 & -1 & 0 \end{pmatrix}$$

□

Ejemplo 6.2 Sea una aplicación lineal $f: \mathbb{R}^2 \rightarrow \mathbb{R}^3$ cuya matriz asociada respecto de las bases canónicas de ambos espacios es:

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}$$

Obtener las imágenes mediante la aplicación lineal de los vectores $v_1 = (1, -2)$ y $v_2 = (-2, 4)$.

En primer lugar, introducimos la matriz asociada a la aplicación lineal y posteriormente obtenemos las imágenes.

```
clear all
A = [1 2; 3 4; 5 6]
```

$$A = \begin{matrix} 3 \times 2 \\ \begin{matrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{matrix} \end{matrix}$$

$$v1 = [1 \quad -2], \quad v2 = [-2 \quad 4]$$

$$v1 = \begin{pmatrix} 2x1 \\ 1 \\ -2 \end{pmatrix}$$

$$v2 = \begin{pmatrix} 2x1 \\ -2 \\ 4 \end{pmatrix}$$

$$fv1 = A * v1$$

$$fv1 = \begin{pmatrix} 3x1 \\ -3 \\ -5 \\ -7 \end{pmatrix}$$

$$fv2 = A * v2$$

$$fv2 = \begin{pmatrix} 3x1 \\ 6 \\ 10 \\ 14 \end{pmatrix}$$

□

6.2. Núcleo e imagen de una aplicación lineal

El núcleo de una aplicación lineal o de cualquiera de sus matrices coordenadas es el subespacio

$$\text{Ker}(f) = f^{-1}(0_V) = \{x \in U \mid f(x) = 0_V\} = \{X \mid AX = 0\}$$

es decir, el conjunto de soluciones del sistema homogéneo.

La imagen de una aplicación lineal f es el subespacio

$$\text{Im}f = f(U)$$

es decir, el conjunto de combinaciones lineales de los vectores $f(u_i)$, si $\{u_i, i = 1, \dots, n\}$ es una base cualquiera de U .

En particular, si A es la matriz coordenada de f en ciertas bases, las columnas de A componen un sistema generador de $\text{Im}f$.

Ejemplo 6.3 Sea $f: \mathbb{R}^4 \longrightarrow \mathbb{R}^5$ una aplicación lineal cuya matriz respecto a las bases canónicas es:

$$B = \begin{pmatrix} 1 & -1 & 2 & 5 \\ -1 & 0 & 3 & 2 \\ 4 & -8 & 34 & 52 \\ 2 & -3 & 12 & 19 \\ 0 & -1 & 5 & 7 \end{pmatrix}$$

1. Obtener las dimensiones del núcleo y de la imagen de f y, si es posible, una base de ambos espacios.

2. Analizar si los vectores $(1, -1, 2, 3)$ y $(0, 11, -2, 3)$ pertenecen al núcleo de f .
1. Para obtener las dimensiones del núcleo y de la imagen deberemos buscar una base de ambos espacios. Las columnas de B ya son sistema generador de la imagen de la aplicación f pero para obtener una base ortonormal podemos realizarlo de dos maneras:

mediante orth que nos devuelve una matriz cuyas columnas forman una base ortonormal de la imagen de la aplicación.

mediante rref que devuelve la matriz escalonada cuyas columnas linealmente independientes forman una base de la imagen de la aplicación.

Nota 6.1 Las bases obtenidas están referidas en las bases respecto de las que está asociada la matriz de la aplicación.

```
>> B
```

```
B =
```

1	-1	2	5
-1	0	3	2
4	-8	34	52
2	-3	12	19
0	-1	5	7

```
>> null(B)
```

```
ans =
```

```

0
745/784
-778/4503
389/1501
```

```
>> orth(B)
```

```
ans =
```

-151/1884	1268/2471	4969/8535
-107/2251	-468/601	88/11439
-3277/3530	-124/2283	741/10672
-962/2859	344/1443	-1746/3145
-232/1817	-935/3521	863/1463

```
>> rref(B)
```

```
ans =
```

1	0	0	0
0	1	0	-11/3
0	0	1	2/3
0	0	0	0

0 0 0 0

Por lo tanto podemos decir que la dimensión del núcleo es 1 y la de la imagen es 3.

2. Veamos si los vectores pertenecen al núcleo.

```
>> v1=[1,-1,2,3],v2=[0,11,-2,3]
```

```
v1 =
```

1 -1 2 3

```
v2 =
```

0 11 -2 3

```
>> B*[v1;v2]'
```

```
ans =
```

21 0
11 0
236 0
86 0
32 0

El primero de ellos no pertenece al núcleo y el segundo si.

□

6.3. Cambio de bases en una aplicación lineal

Si A es la matriz coordenada de la aplicación lineal f respecto de las bases $\{u_1, \dots, u_n\}$ y $\{v_1, \dots, v_m\}$ de U y V , respectivamente, y B es la matriz coordenada de la misma aplicación respecto de otras bases $\{u'_1, \dots, u'_n\}$ y $\{v'_1, \dots, v'_m\}$, la relación existente entre ellas es

$$A \cdot P = Q \cdot B \text{ o bien } A = Q \cdot B \cdot P^{-1}$$

siendo P la matriz que define el cambio de la base $\{u_1, \dots, u_n\}$ a la base $\{u'_1, \dots, u'_n\}$ y Q la matriz que define el cambio de la base $\{v_1, \dots, v_m\}$ a la $\{v'_1, \dots, v'_m\}$.

Ejemplo 6.4 Obtener la matriz asociada a la aplicación lineal del ejemplo 6.3 respecto de las bases

$$\begin{aligned} U &= \{(3, 0, 1, -5), (2, -1, 0, 1), (-2, 3, 4, 2), (1, 0, 2, 0)\} \\ V &= \{(1, 1, 1, 1, 1), (0, 1, 1, 1, 1), (0, 0, 1, 1, 1), (0, 0, 0, 1, 1), (0, 0, 0, 0, 1)\}. \end{aligned}$$

En primer lugar, escribimos las matrices de cambio de base y de la aplicación que tenemos almacenada del ejemplo anterior.

```
>> P=[3,0,1,-5;2,-1,0,1;-2,3,4,2;1,0,2,0] '
P =
```

3	2	-2	1
0	-1	3	0
1	0	4	2
-5	1	2	0

```
>> Q=[1,1,1,1,1,1;0,1,1,1,1,1;0,0,1,1,1,1;0,0,0,1,1,1;0,0,0,0,1,1] '
Q =
```

1	0	0	0	0
1	1	0	0	0
1	1	1	0	0
1	1	1	1	0
1	1	1	1	1

```
>> B
B =
```

1	-1	2	5
-1	0	3	2
4	-8	34	52
2	-3	12	19
0	-1	5	7

Ahora, obtenemos la matriz asociada a la aplicación respecto a las bases U y V .

```
>> A=Q^-1*B*P
A =
```

-20	8	13	5
10	-8	5	0
-204	68	190	67
137	-42	-135	-46
47	-18	-42	-16

□

Ejemplo 6.5 Sea $f: \mathbb{R}^3 \longrightarrow \mathbb{R}^3$ un endomorfismo definido por

$$f(u_1) = u_1, f(u_2) = 2u_2, f(u_3) = -u_3,$$

donde u_1, u_2 y u_3 son vectores de $\mathbb{R}^3$ cuyas coordenadas respecto de la base canónica son:

$$u_1 = (-1, 0, 2), u_2 = (0, 1, 7), u_3 = (2, 0, 3).$$

Se pide:

1. Comprobar que $B = \{u_1, u_2, u_3\}$ es una base de $\mathbb{R}^3$.
2. Hallar la matriz asociada al endomorfismo f respecto de la base canónica.
3. Hallar la imagen por f del vector $v = 2u_1 + u_2$ y expresarla en la base canónica y en la base B .

4. Probar que f es biyectiva. Hallar el vector cuya imagen por f es $3u_1 + u_2$ y expresarlo en la base canónica.

1. Introducimos la base por filas y calculamos, por ejemplo, su rango

```
>> B=[-1,0,2;0,1,7;2,0,3]
```

```
B =
```

-1	0	2
0	1	7
2	0	3

```
>> rank(B)
```

```
ans =
```

```
3
```

como es 3 B es una base de $\mathbb{R}^3$

2. La matriz asociada a f respecto a la base B es:

```
>> M=[1,0,0;0,2,0;0,0,-1]
```

```
M =
```

1	0	0
0	2	0
0	0	-1

y las matrices de cambio de base son en ambos casos:

```
>> P=B'
```

```
P =
```

-1	0	2
0	1	0
2	7	3

luego la matriz asociada a f respecto de la base canónica es:

```
>> Mc=P*M*P^-1
```

```
Mc =
```

-1/7	4	-4/7
0	2	0
-12/7	13	1/7

3. Para obtener la imagen de $v = 2u_1 + u_2$ podemos realizar varios procesos pero si queremos utilizar las matrices asociadas en ambos casos procederemos así:

```

>> u1=B(1,:)

u1 =

    -1         0         2

>> u2=B(2,:)

u2 =

     0         1         7

>> u3=B(3,:)

u3 =

     2         0         3

>> vB=[2 1 0]% respecto de la base B

vB =

     2         1         0

>> vc=2*u1+u2% respecto de la base canónica

vc =

    -2         1        11

>> M*vB'% respecto de la base B

ans =

     2
     2
     0

>> Mc*vc'% respecto de la base canónica

ans =

    -2
     2
    18

```

4. *Para ver si es biyectiva, dado que es un endomorfismo, basta ver que el determinante de una matriz asociada a la aplicación f es no nulo, es decir, que su rango es máximo.*

```

>> det(Mc)

```

```
ans =
```

```
-2
```

y para obtener la antiimagen del vector $u = 3u_1 + u_2$ basta multiplicar el vector por la matriz inversa de la asociada de f que es la matriz asociada a la aplicación lineal inversa de la f

```
>> u=3*u1+u2%respecto de la base canónica
```

```
u =
```

```
-3          1          13
```

```
>> Mc^-1*u'
```

```
ans =
```

```
-3  
1/2  
19/2
```

□

Capítulo 7

Diagonalización

Puesto que una matriz A está asociada a una aplicación lineal f , se plantea la pregunta de si es posible representar f mediante una matriz más sencilla, diagonal. Habrá que determinar, por tanto, si existen bases adecuadas en los espacios de partida y de llegada respecto de las cuales la matriz asociada sea diagonal.

En este capítulo, se abordará el problema de la diagonalización de una matriz partiendo de los conceptos de valor propio y subespacio de vectores propios correspondiente y teniendo en cuenta la interpretación de las columnas de una matriz asociada a una aplicación lineal.

También veremos, como aplicación, la resolución de sistemas de ecuaciones diferenciales.

En lo sucesivo, salvo que se diga lo contrario, A es una matriz cuadrada.

Las órdenes usuales para resolver estos problemas son:

Orden	Salida
<code>eig(A)</code>	Devuelve un vector columna que contiene los valores propios de la matriz cuadrada A .
<code>[P,D]=eig(A)</code>	Devuelve una matriz diagonal D cuyos elementos diagonales son los valores propios de la matriz cuadrada A y una matriz P cuyas columnas son los correspondientes vectores propios, tales que $AP=PD$.
<code>poly(A)</code>	Devuelve un vector fila con $N+1$ coordenadas que son los coeficientes del polinomio característico $\det(\lambda I - A)$, donde N es el tamaño de la matriz cuadrada A .
<code>poly(v)</code>	Devuelve un vector cuyas coordenadas son los coeficientes del polinomio cuyas raíces son las coordenadas del vector v .
<code>roots(p)</code>	Devuelve un vector cuyas coordenadas son las raíces del polinomio p .
<code>limit(F,x,a)</code>	Calcula el límite de la expresión simbólica F cuando $x \rightarrow a$.

7.1. Valores y vectores propios

Sea $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ una aplicación lineal.

Se dice que un vector $\vec{x} \neq \vec{0} \in \mathbb{R}^n$ es un vector propio o autovector de f si existe $\lambda \in \mathbb{R}$ tal que:
 $f(\vec{x}) = \lambda \vec{x}$

Al escalar λ , que, si existe, es único, se le conoce como valor propio o autovalor asociado al vector propio $\vec{x}$.

Sea A la matriz asociada a determinada aplicación lineal f en una cierta base $\mathcal{B}$ de $\mathbb{R}^n$, sea I la matriz identidad de orden n , y sea X la matriz columna de las coordenadas del vector $\vec{x}$ en la base $\mathcal{B}$. Entonces la condición de vector propio se escribe:

$$AX = \lambda I_n X \text{ o, equivalentemente, } (A - \lambda I_n)X = 0$$

Esta ecuación matricial equivale a un sistema lineal homogéneo de n ecuaciones en las n incógnitas que corresponden a las n coordenadas de $\vec{x}$. Los vectores propios asociados a λ serán las soluciones no triviales de este sistema, que admite alguna de estas soluciones si y sólo si

$$|A - \lambda I_n| = 0$$

El determinante anterior genera, al desarrollarlo, un polinomio en λ de grado n , que se denomina *polinomio característico* de la matriz A , y la ecuación obtenida al igualarlo a 0, se llama la *ecuación característica* de la aplicación lineal f o de la matriz A .

El conjunto de vectores anterior se denomina *subespacio fundamental* o subespacio propio asociado al valor propio λ .

Ejemplo 7.1 *Obténgase autovalores y vectores propios de la matriz*

$$A = \begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{pmatrix}$$

En primer lugar, para obtener los autovalores, vamos a plantear y a resolver la ecuación característica.

```
>> A=[1 1 0 0 ;0 1 1 0 ;1 0 0 1;0 0 1 1 ]
```

```
A =
```

```

1      1      0      0
0      1      1      0
1      0      0      1
0      0      1      1
```

```
>> syms x
```

```
>> B=A-x*eye(4)
```

```
B =
```

```
[ 1 - x,      1,  0,      0]
```

```
[      0, 1 - x,  1,      0]
```

```
[      1,      0, -x,      1]
```

```
[      0,      0,  1, 1 - x]
```

```
>> p=det(B)
```

```
p =
```

```
x^4 - 3*x^3 + 2*x^2
```

```
>> factor(p)
```

```
ans =
```

```
x^2*(x - 1)*(x - 2)
```

Con lo que se tienen los autovalores $x = 0$, $x = 1$ y $x = 2$. El primero de ellos con multiplicidad 2 y los otros con multiplicidad 1.

Con MATLAB lo habríamos realizado con una sola orden: la orden `eig` que devuelve

```
>> [Q D]=eig(A)
```

```
Q =
```

```
-0.5000      0.5000 + 0.0000i    0.5000 - 0.0000i   -0.7071
-0.5000     -0.5000 - 0.0000i   -0.5000 + 0.0000i    0.0000
-0.5000      0.5000              0.5000             -0.0000
-0.5000     -0.5000 - 0.0000i   -0.5000 + 0.0000i    0.7071
```

```
D =
```

```
2.0000      0      0      0
      0     -0.0000 + 0.0000i      0      0
      0      0     -0.0000 - 0.0000i      0
      0      0      0      1.0000
```

La diagonal de la matriz D está formada por los autovalores y las columnas de la matriz Q son los vectores propios asociados a los autovalores. $\square$

7.1.1. Propiedades básicas

1. Cada vector propio está asociado a un único valor propio, sin embargo, en general, existirán distintos vectores propios asociados a un mismo valor propio.
2. Si $\vec{x}$ es un vector propio de la aplicación lineal f , asociado al valor propio λ , también lo es el vector $\mu\vec{x}$, asociado al mismo valor propio, donde μ es un escalar cualquiera no nulo.
3. Si $\vec{x}$ y $\vec{x}'$ son dos vectores propios asociados al mismo valor propio λ , toda combinación lineal de dichos vectores también es un vector propio asociado al mismo autovalor.
4. Si $\vec{x}_1, \dots, \vec{x}_h$ son h vectores propios asociados a h valores propios distintos, entonces $\{\vec{x}_1, \dots, \vec{x}_h\}$ es un sistema libre.
5. La suma de los valores propios de una matriz A es igual a su traza: $\lambda_1 + \lambda_2 + \dots + \lambda_n = \text{tr}(A)$.

6. El producto de los n valores propios de la matriz A es igual a su determinante: $\lambda_1 \cdot \lambda_2 \cdot \dots \cdot \lambda_n = \det(A)$.

Sólo vamos a comprobar, con el primer autovalor y su vector propio asociado del ejemplo anterior 7.1, las cuatro primeras propiedades.

Ejemplo 7.2

```
>> v1=Q(:,1)
```

```
v1 =
```

```
-0.5000
-0.5000
-0.5000
-0.5000
```

```
>> A*v1
```

```
ans =
```

```
-1.0000
-1.0000
-1.0000
-1.0000
```

```
>> 2*v1
```

```
ans =
```

```
-1.0000
-1.0000
-1.0000
-1.0000
```

*Y es obvio que $A * v1 \neq (0,0,0,0) = 0 * v1$ y $A * v1 \neq 1 * v1$.*

```
>> syms m n
```

```
>> 2*(m*v1+n*[-1 -1 -1 -1]') % [-1 -1 -1 -1] es vector propio asociado a 2
```

```
ans =
```

```
- m - 2*n
- m - 2*n
- m - 2*n
- m - 2*n
```

```
>> A*(m*v1+n*[-1 -1 -1 -1]')
```

```
ans =
```

```
- m - 2*n
- m - 2*n
- m - 2*n
- m - 2*n
```

Veamos ahora la cuarta de las propiedades:

```
>> Q
```

```
Q =
```

```
-0.5000      0.5000 + 0.0000i    0.5000 - 0.0000i   -0.7071
-0.5000     -0.5000 - 0.0000i   -0.5000 + 0.0000i    0.0000
-0.5000      0.5000              0.5000              -0.0000
-0.5000     -0.5000 - 0.0000i   -0.5000 + 0.0000i    0.7071
```

```
>> Q(:,3)=[] %eliminamos la tercera columna
```

```
Q =
```

```
-0.5000      0.5000 + 0.0000i   -0.7071
-0.5000     -0.5000 - 0.0000i    0.0000
-0.5000      0.5000              -0.0000
-0.5000     -0.5000 - 0.0000i    0.7071
```

```
>> rref(Q)
```

```
ans =
```

```
1.0000      0      0
      0      1.0000      0
      0      0      1.0000 - 0.0000i
      0      0      0
```

Lo hemos comprobado para tres vectores propios ya que tenemos tres autovalores.

□

Veamos, con el mismo ejemplo 7.1, las otras dos propiedades.

Ejemplo 7.3 >> autovalores=eig(A)

```
autovalores =
```

```
2.0000
-0.0000 + 0.0000i
-0.0000 - 0.0000i
1.0000
```

```
>> sum(autovalores)
```

```

ans =

      3
>> trace(A)

ans =

      3

>> det(A)

ans =

      0

>> prod(autovalores)
1

ans =

 9.9033e-017

```

□

7.2. Diagonalización de endomorfismos y matrices

Dos matrices cuadradas A y B sobre $\mathbb{R}$ ó $\mathbb{C}$ se dicen semejantes si existe una matriz regular P tal que $P^{-1}AP = B$, o bien, equivalentemente, si $AP = PB$. Observar que las matrices A y B corresponden al mismo endomorfismo en dos bases relacionadas por la matriz de cambio P .

El determinante y la traza se conservan por semejanza, es decir, si A y B son matrices semejantes, entonces $|A| = |B|$ y $tr(A) = tr(B)$. Luego podemos afirmar que la traza y el determinante son invariantes por cambios de base, lo que pone de manifiesto su naturaleza geométrica.

Una matriz cuadrada A de orden n se dice *diagonalizable* si se puede encontrar una base de $\mathbb{R}^n$ de vectores propios o, equivalentemente, si la suma de dimensiones de los subespacios fundamentales (multiplicidades geométricas) es igual a n .

El polinomio característico de una matriz A asociada a una aplicación lineal f de $\mathbb{R}^n$ en $\mathbb{R}^n$, resulta invariante ante un cambio de base en $\mathbb{R}^n$.

Ejemplo 7.4 Dada la transformación lineal

$$\begin{aligned}
 f: \quad \mathbb{R}^4 &\longrightarrow \mathbb{R}^4 \\
 (x, y, z, t) &\longrightarrow (3x + 2y + z, 4y + t, 2z, t)
 \end{aligned}$$

obténanse sus autovalores y los vectores propios asociados a dichos autovalores y compruébese que son linealmente independientes.

¹El valor del producto no es exactamente 0 debido a los errores de redondeo de la máquina.

Además, compruébese, también, que la traza y el determinante, así como el polinomio característico son invariantes por cambio de base.

En primer lugar debemos introducir la matriz asociada a la transformación lineal para, posteriormente, obtener los autovalores y los autovectores.

```
>> A=[3 0 0 0; 2 4 0 0; 1 0 2 0;0 1 0 1]'
```

A =

3	2	1	0
0	4	0	1
0	0	2	0
0	0	0	1

```
>> [V D]=eig(A)
```

V =

1.0000	0.8944	-0.7071	0.3015
0	0.4472	0	-0.3015
0	0	0.7071	0
0	0	0	0.9045

D =

3	0	0	0
0	4	0	0
0	0	2	0
0	0	0	1

Se tienen 4 autovalores diferentes y sus vectores asociados, obviamente son independientes puesto que el rango de la matriz V es, claramente, 4.

De forma mas elegante podemos realizar la siguiente secuencia de comandos de MATLAB:

```
>> p=poly(A)
```

p =

1	-10	35	-50	24
---	-----	----	-----	----

```
>> roots(p)
```

ans =

4.0000
3.0000
2.0000
1.0000

O, incluso podemos obtener el polinomio característico en forma simbólica y luego sus raíces.

```
>> poly(sym(A))

ans =

x^4 - 10*x^3 + 35*x^2 - 50*x + 24

>> solve(ans)

ans =

1
2
3
4
```

A la vista de los resultados obtenidos podemos afirmar que la matriz es diagonalizable ya que los 4 subespacios propios tienen dimensión 1. También se puede observar que los 4 vectores propios asociados a los 4 autovalores distintos forman base del espacio $\mathbb{R}^4$.

Para la comprobación de los invariantes sólo debemos compararlos para las matrices A y D.

```
>> det(D)==det(A), trace(D)==trace(A), poly(D)==poly(A)

ans =

1

ans =

1

ans =

1      1      1      1      1
```

Como vemos todos ellos coinciden. □

Podemos encontrarnos, como en el siguiente ejemplo, con matrices o endomorfismos con algunos autovalores iguales.

Ejemplo 7.5 Sea la matriz

$$A = \begin{pmatrix} 18 & -15 & -25 & 0 \\ 0 & -2 & 0 & 0 \\ 0 & 0 & -2 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Obténganse sus autovalores con sus multiplicidades algebraicas y sus vectores propios asociados.

```
>> A =[ 18 -15 -25 0; 0 -2 0 0; 0 0 -2 0; 0 0 0 1]
```

```
A =
```

```
18    -15   -25    0
 0     -2     0    0
 0      0    -2    0
 0      0     0    1
```

```
>> [V D]=eig(A)
```

```
V =
```

```
1.0000    0.6000    0.7809         0
         0    0.8000         0         0
         0         0    0.6247         0
         0         0         0    1.0000
```

```
D =
```

```
18     0     0     0
 0    -2     0     0
 0     0    -2     0
 0     0     0     1
```

Observamos que tenemos dos autovalores iguales, $\lambda = -2$, por lo que la multiplicidad algebraica es 2 y la de los otros autovalores es 1. Podemos calcular las multiplicidades geométricas para determinar si la matriz es diagonalizable.

```
>> 4-rank(A-18*eye(4))
```

```
ans =
```

```
1
```

```
>> 4-rank(A+2*eye(4))
```

```
ans =
```

```
2
```

```
>> 4-rank(A-1*eye(4))
```

```
ans =
```

```
1
```

Así, vemos que las multiplicidades algebraicas coinciden con las geométricas y por tanto la matriz A es diagonalizable.

Por otro lado, las bases de los subespacios propios se pueden calcular buscando el núcleo de $A - \lambda Id$ aunque ya las columnas de la matriz V son, precisamente, vectores que forman base de los espacios propios.

```
>> null(A-1*eye(4))
```

```
ans =
```

```
0
0
0
1
```

```
>> null(A-18*eye(4))
```

```
ans =
```

```
-1.0000
0
0
0
```

```
>> null(A+2*eye(4))
```

```
ans =
```

```
-0.8246    0
-0.2910   -0.8575
-0.4851    0.5145
0          0
```

□

Ejemplo 7.6 Sea la matriz

$$A = \begin{pmatrix} 5 & 2 & 0 & 1 \\ -2 & 1 & 0 & -1 \\ 4 & 4 & 3 & 2 \\ 16 & 0 & -8 & -5 \end{pmatrix}$$

Compruébese que la matriz no es diagonalizable.

Podemos iniciar la comprobación calculando las raíces del polinomio característico y viendo cuales son sus multiplicidades.

```
>> A=[5,2,0,1;-2,1,0,-1;4,4,3,2;16,0,-8,-5]
```

```
A =
```

```
5          2          0          1
-2         1          0         -1
4          4          3          2
```

16 0 -8 -5

```
>> solve(poly(sym(A)))
```

ans =

-5
3
3
3

Los autovalores son -5 con multiplicidad algebraica 1 y 3 con multiplicidad 3.

Podemos obtener sus vectores propios con la orden eig.

```
>> [V D]=eig(A)
```

V =

0.1195	0.4082	-0.4082	-0.2717
-0.1195	-0.4082	0.4082	0.6270
0.2390	0.8165	-0.8165	0.1673
-0.9562	0.0000	0.0000	-0.7107

D =

-5.0000	0	0	0
0	3.0000	0	0
0	0	3.0000	0
0	0	0	3.0000

Como vemos, las columnas 2 y 3 de la matriz V son linealmente dependientes por lo que la matriz no va a ser diagonalizable ya que la multiplicidad geométrica del autovalor 3 va a ser 2.

Comprobemos esta afirmación calculando las multiplicidades algebraicas y geométricas de los autovalores.

```
>> 4-rank(A+5*eye(4))
```

ans =

1

```
>> 4-rank(A-3*eye(4))
```

ans =

2

7.2.1. Aplicaciones de la diagonalización

La diagonalización de matrices tiene múltiples aplicaciones. Aquí veremos sólo algunas de ellas como el cálculo de inversas y potencias de matrices o su aplicación a la resolución de sistemas de ecuaciones diferenciales.

Potencias de matrices diagonalizables

Si una matriz A es diagonalizable, es decir, $A = P \cdot D \cdot P^{-1}$, se comprueba fácilmente que

$$A^n = P \cdot D^n \cdot P^{-1}$$

Esta afirmación también es válida para $n \in \mathbb{Z}^-$.

Ejemplo 7.7 Dada la matriz

$$A = \begin{pmatrix} 9 & -3 & -11 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & -2 & 0 \\ 0 & 3 & 0 & -1 \end{pmatrix}$$

Obtener su inversa, A^{-5} y A^4 .

```
>> A =[ 9 -3 -11 0; 0 1 1 0; 0 0 -2 0; 0 3 0 -1]
```

```
A =
```

```

9      -3     -11      0
0       1       1      0
0       0      -2      0
0       3       0     -1
```

```
>> [V D]=eig(A)
```

```
V =
```

```

1.0000      0    0.2037    0.5304
      0      0    0.5431   -0.1945
      0      0      0     0.5835
      0    1.0000    0.8146    0.5835
```

```
D =
```

```

9      0      0      0
0     -1      0      0
0      0      1      0
0      0      0     -2
```

```
>> A^-1
```

ans =

0.1111	0.3333	-0.4444	0
0	1.0000	0.5000	0
0	0	-0.5000	0
0	3.0000	1.5000	-1.0000

>> V*D⁻¹*V⁻¹

ans =

0.1111	0.3333	-0.4444	0
0	1.0000	0.5000	0
0	0	-0.5000	0
0	3.0000	1.5000	-1.0000

>> A⁻⁵

ans =

0.0000	0.3750	0.0966	0
0	1.0000	0.3438	0
0	0	-0.0313	0
0	3.0000	1.9688	-1.0000

>> V*D⁻⁵*V⁻¹

ans =

0.0000	0.3750	0.0966	0
0	1.0000	0.3438	0
0	0	-0.0313	0
0	3.0000	1.9688	-1.0000

>> A⁴

ans =

6561	-2460	-6770	0
0	1	-5	0
0	0	16	0
0	0	15	1

>> V*D⁴*V⁻¹

ans =

1.0e+003 *

6.5610	-2.4600	-6.7700	0
--------	---------	---------	---

$$\begin{array}{cccc} 0 & 0.0010 & -0.0050 & 0 \\ 0 & 0 & 0.0160 & 0 \\ 0 & 0 & 0.0150 & 0.0010 \end{array}$$

□

Aplicación a los sistemas de ecuaciones diferenciales

En muchos problemas de aplicación, varias cantidades varían continuamente con el tiempo y están relacionados por medio de un sistema de n ecuaciones diferenciales con n funciones desconocidas (incógnitas) definidas mediante:

$$\begin{cases} x'_1(t) = a_{11}x_1(t) + a_{12}x_2(t) + \dots + a_{1n}x_n(t) \\ x'_2(t) = a_{21}x_1(t) + a_{22}x_2(t) + \dots + a_{2n}x_n(t) \\ \vdots \\ x'_n(t) = a_{n1}x_1(t) + a_{n2}x_2(t) + \dots + a_{nn}x_n(t) \end{cases}$$

que se puede expresar matricialmente como:

$$x'(t) = A \cdot x(t)$$

siendo $x(t)$ una función vectorial.

El caso mas sencillo de estos sistemas es el que consiste en una sola ecuación diferencial:

$$x'(t) = a \cdot x(t)$$

que para ser resuelta se multiplica ambos miembros por el recíproco de $x(t)$ y se integra respecto de t , quedando:

$$\int \frac{x'(t)}{x(t)} dt = \int a dt \implies \ln(|x(t)|) = a \cdot t + c \implies |x(t)| = e^{at} e^c \implies x(t) = k \cdot e^{at} \text{ con } k \text{ real.}$$

que es la solución general de la ecuación diferencial.

Si en el sistema en forma matricial se considera A diagonal, entonces el sistema inicial se reduce a un sistema de n ecuaciones diferenciales, es decir:

$$\begin{cases} x'_1(t) = a_1 x_1(t) \\ x'_2(t) = a_2 x_2(t) \\ \vdots \\ x'_n(t) = a_n x_n(t) \end{cases}$$

cuya solución general es:

$$x(t) = \begin{pmatrix} x_1(t) \\ x_2(t) \\ \vdots \\ x_n(t) \end{pmatrix} = \begin{pmatrix} k_1 \cdot e^{a_1 t} \\ k_2 \cdot e^{a_2 t} \\ \vdots \\ k_n \cdot e^{a_n t} \end{pmatrix}$$

Por lo tanto vamos a intentar reducir el sistema de ecuaciones diferenciales a uno de tipo diagonal mediante la diagonalización de la matriz A .

Si A es diagonalizable existe una matriz Q regular tal que $A = Q \cdot D \cdot Q^{-1}$, entonces el sistema queda

$$x'(t) = Q \cdot D \cdot Q^{-1} \cdot x(t) \implies Q^{-1} \cdot x'(t) = D \cdot Q^{-1} \cdot x(t)$$

y haciendo $y(t) = Q^{-1} \cdot x(t)$ tenemos un nuevo sistema que es diagonal

$$y'(t) = D \cdot y(t)$$

cuya solución es, como hemos visto anteriormente,

$$y(t) = \begin{pmatrix} y_1(t) \\ y_2(t) \\ \vdots \\ y_n(t) \end{pmatrix} = \begin{pmatrix} k_1 \cdot e^{\lambda_1 t} \\ k_2 \cdot e^{\lambda_2 t} \\ \vdots \\ k_n \cdot e^{\lambda_n t} \end{pmatrix}$$

donde λ_i son los autovalores asociados a la matriz A y que forman la diagonal de la matriz D .

Finalmente, la solución del sistema inicial $x'(t) = A \cdot x(t)$ es

$$x(t) = Q \cdot y(t)$$

donde Q es la matriz, por columnas, de los autovalores de la matriz A .

Nota 7.1 *El proceso descrito sólo es válido si A es diagonalizable; en tal caso los autovalores no necesitan ser distintos en tanto que la multiplicidad algebraica y geométrica resulten iguales.*

Ejemplo 7.8 *Resuélvase el siguiente sistema de ecuaciones diferenciales:*

$$\begin{cases} x'_1 &= x_1 - x_2 - x_3 \\ x'_2 &= -x_1 + x_2 - x_3 \\ x'_3 &= -x_1 - x_2 + x_3 \end{cases}$$

En primer lugar introducimos la matriz de coeficientes del sistema de ecuaciones y calculamos sus autovectores y autovalores para comprobar si es A es diagonalizable:

```
>> A=sym([1,-1,-1;-1,1,-1;-1,-1,1])
```

```
A =
```

```
[ 1, -1, -1]
[-1,  1, -1]
[-1, -1,  1]
```

```
>> [Q D]=eig(A)
```

```
Q =
```

```
[ 1, -1, -1]
[ 1,  1,  0]
[ 1,  0,  1]
```

D =

$$\begin{bmatrix} -1, & 0, & 0 \\ 0, & 2, & 0 \\ 0, & 0, & 2 \end{bmatrix}$$

Lo es, por tanto podemos decir que la solución del sistema $x'(t) = Q \cdot y'(t)$ que se obtiene haciendo $x(t) = Q \cdot y(t)$ es:

$$\begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} k_1 \cdot e^{-t} \\ k_2 \cdot e^{2t} \\ k_3 \cdot e^{2t} \end{pmatrix}$$

y, por tanto, la solución del sistema original es $x(t) = Q \cdot y(t)$, es decir,

$$\begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 1 & -1 & -1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix} \begin{pmatrix} k_1 \cdot e^{-t} \\ k_2 \cdot e^{2t} \\ k_3 \cdot e^{2t} \end{pmatrix}$$

$$\begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} k_1 \cdot e^{-t} - k_2 \cdot e^{2t} - k_3 \cdot e^{2t} \\ k_1 \cdot e^{-t} + k_2 \cdot e^{2t} \\ k_1 \cdot e^{-t} + k_3 \cdot e^{2t} \end{pmatrix}$$

Si además añadimos las condiciones iniciales $x_1(0) = 1$, $x_2(0) = 0$ y $x_3(0) = 1$ podemos obtener el valor de las constantes k_1 , k_2 y k_3 obteniendo así la solución

$$\begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} \frac{2}{3} \cdot e^{-t} + \frac{1}{3} \cdot e^{2t} \\ \frac{2}{3} \cdot e^{-t} - \frac{2}{3} \cdot e^{2t} \\ \frac{2}{3} \cdot e^{-t} + \frac{1}{3} \cdot e^{2t} \end{pmatrix}$$

□

Nota 7.2 Recogemos a continuación las órdenes básicas de MATLAB para la resolución de sistemas de ecuaciones diferenciales.

Orden	Descripción
<code>dsolve(ec1,ec2,...,'x')</code>	Devuelve la solución general del sistema de ecuaciones diferenciales de variable independiente x
<code>dsolve(ec1,ec2,...,cond1,cond2,..., 'x')</code>	Devuelve la solución del sistema respecto de la variable independiente x verificando las condiciones iniciales indicadas

Una ecuación diferencial la escribiremos siempre entre comillas simples. En MATLAB, y' se representa por `Dy`, y'' se representa por `D2y`, y''' por `D3y`, etc.

Así la ecuación $\frac{d^3y}{dt^3} + 4\frac{dy}{dt} = \sin^2 t$ se escribiría en MATLAB como `'D3y+4*Dy=sin(t)^2'`.

Las condiciones iniciales también van entre comillas simples. Por ejemplo, las condiciones $y(0) = 1$, $y'(0) = 2$, $y''(0) = 3$ se escriben `'y(0) = 1', 'Dy(0) = 2', 'D2y(0) = 3'`.

Ejemplo 7.9 Si intentamos resolver el sistema del ejemplo anterior 7.8 con las condiciones iniciales expuestas utilizando órdenes de MATLAB deberíamos realizar las siguientes órdenes

```
>> sol=dsolve('Dx=x-y-z','Dy=-x+y-z','Dz=-x-y+z',  
'x(0)=1','y(0)=0','z(0)=1','t')
```

```
sol =
```

```
    x: [1x1 sym]  
    y: [1x1 sym]  
    z: [1x1 sym]
```

```
>> sol.x,sol.y,sol.z
```

```
ans =
```

```
2/(3*exp(t)) + exp(2*t)/3
```

```
ans =
```

```
2/(3*exp(t)) - (2*exp(2*t))/3
```

```
ans =
```

```
2/(3*exp(t)) + exp(2*t)/3
```

□

Parte VI

PRUEBA

